

# B/B+ Trees

*Dr.Kumkum Saxena*

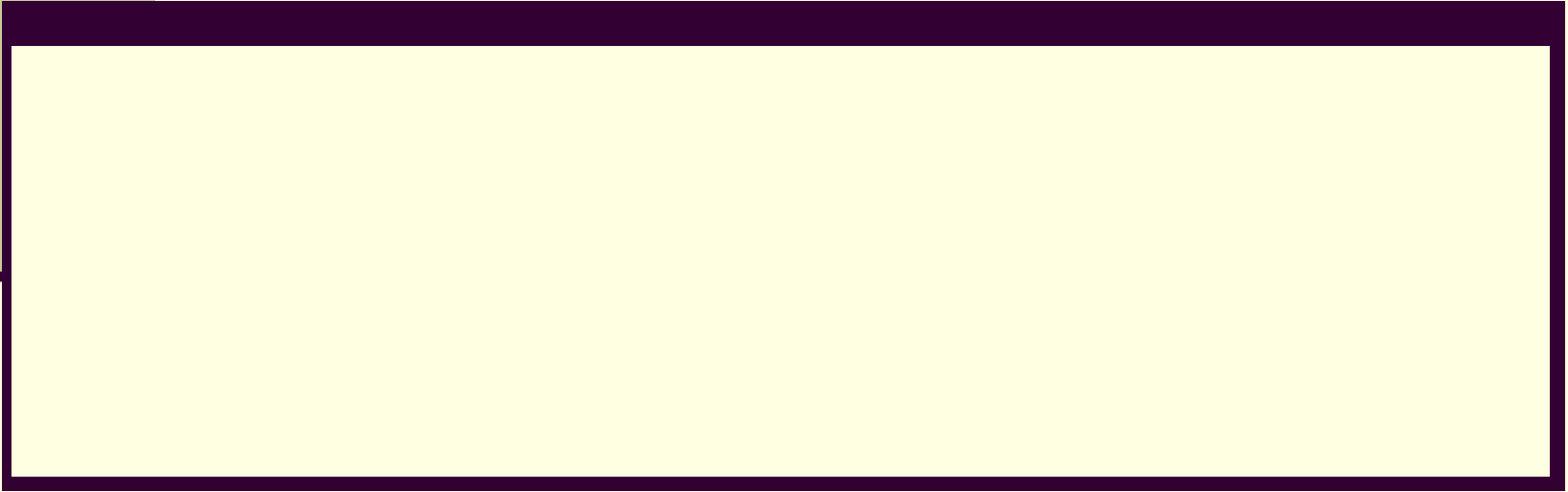
# Indexing

---

- Three main techniques
  - Conventional indexes
    - Think of a page table, ...
  - B and B+ trees
    - Perform better when records are constantly added or deleted
  - Hashing



# Conventional indexes



# Indexes

---

- A **database index** is a data structure that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space to maintain the index data structure.

*Wikipedia*

# Types of indexes

---

- An index can be

- **Sparse**

- One entry per data block
    - Identifies the first record of the block
    - Requires data to be sorted

- **Dense**

- One entry per record
    - Data do not have to be sorted

# Respective advantages

---

## ■ **Sparse**

- Occupy much less space
- Can keep more of it in main memory
  - *Faster access*

## ■ **Dense**

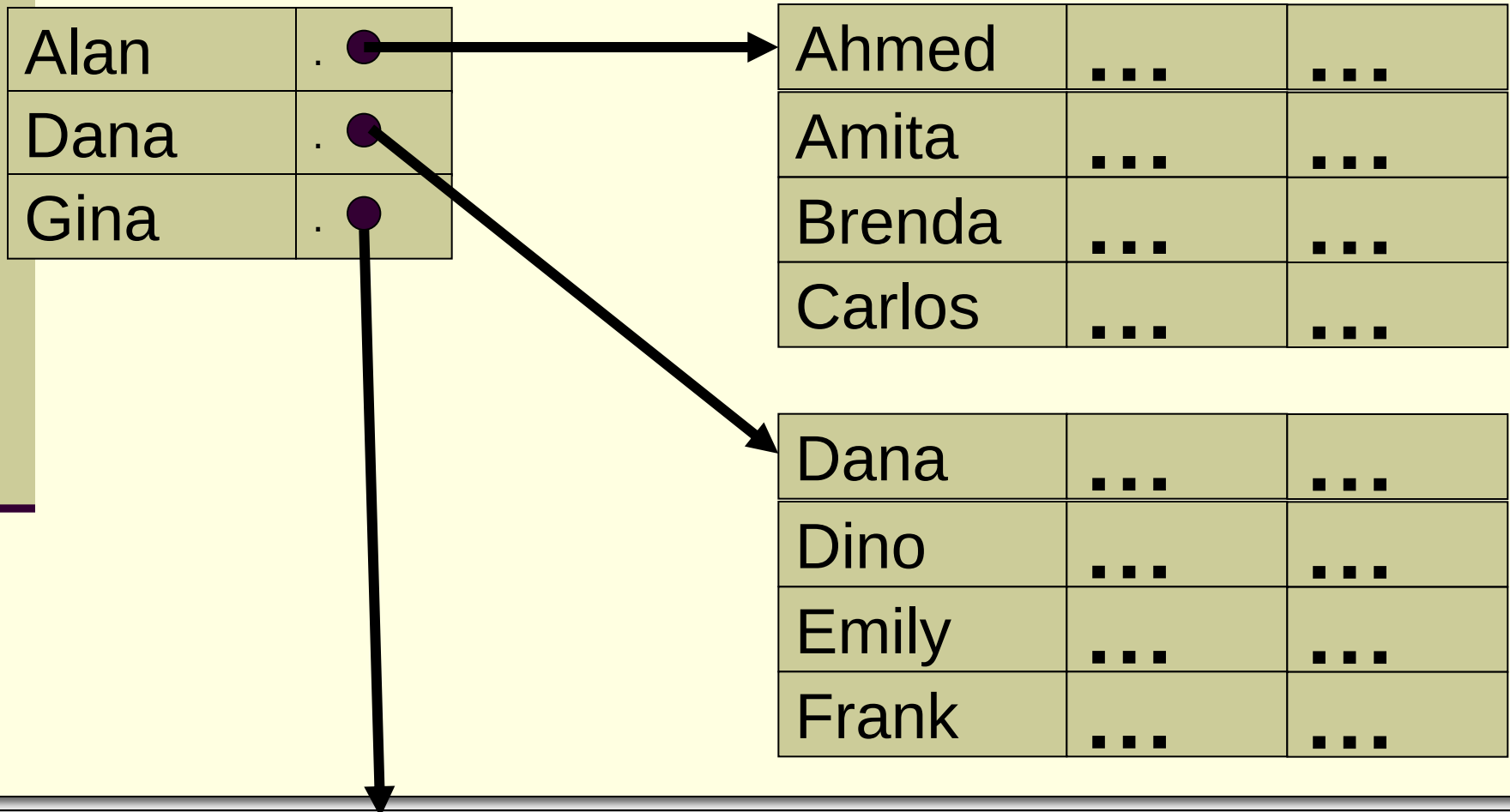
- Can tell if a given record exists without accessing the file
- Do not require data to be sorted

# Indexes based on primary keys

---

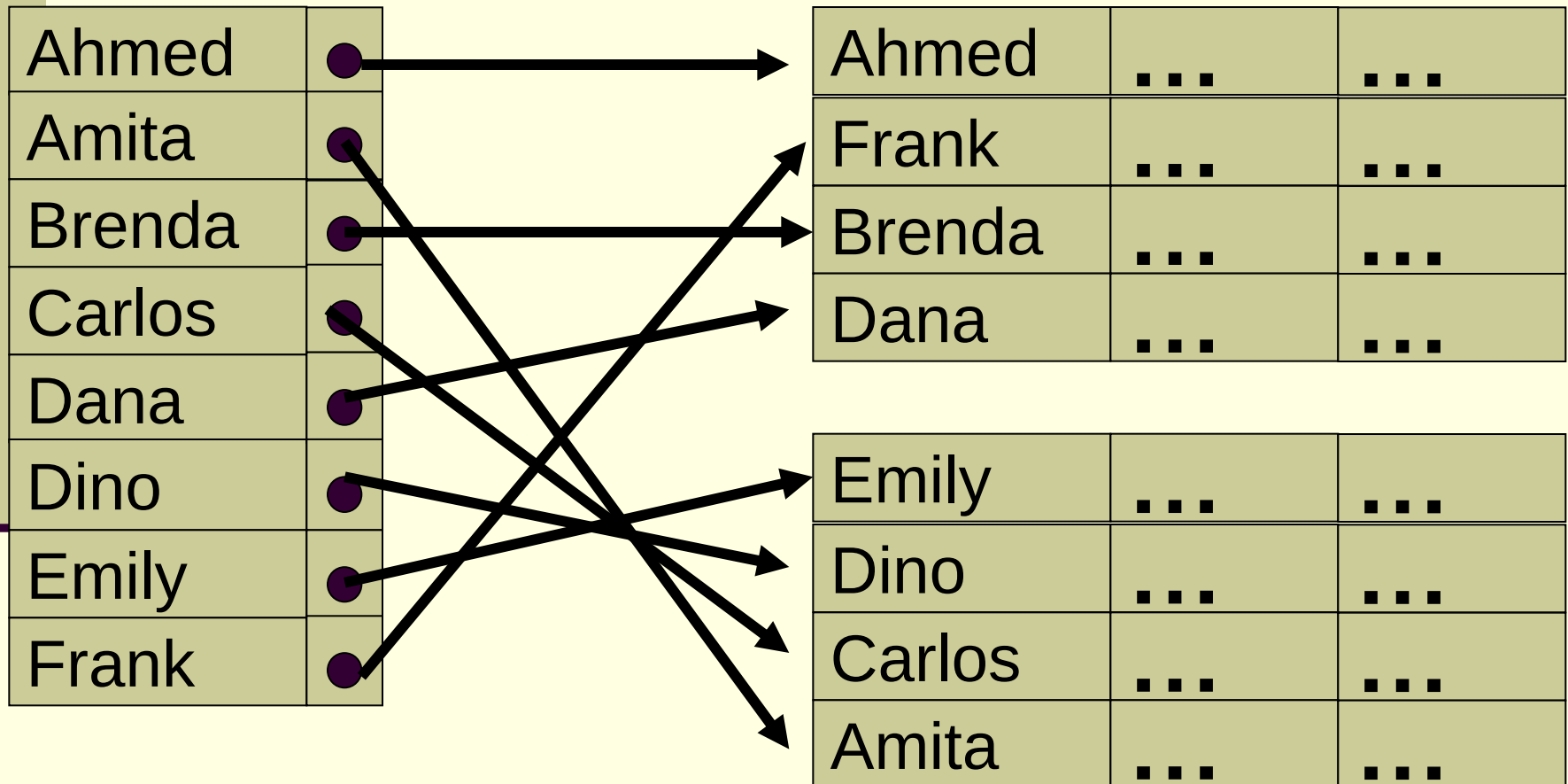
- Each key value corresponds to a specific record
- Two cases to consider:
  - Table is sorted on its primary key
    - Can use a sparse index
  - Table is either non-sorted or sorted on another field
    - Must use a dense index

# Sparse Index





# Dense Index

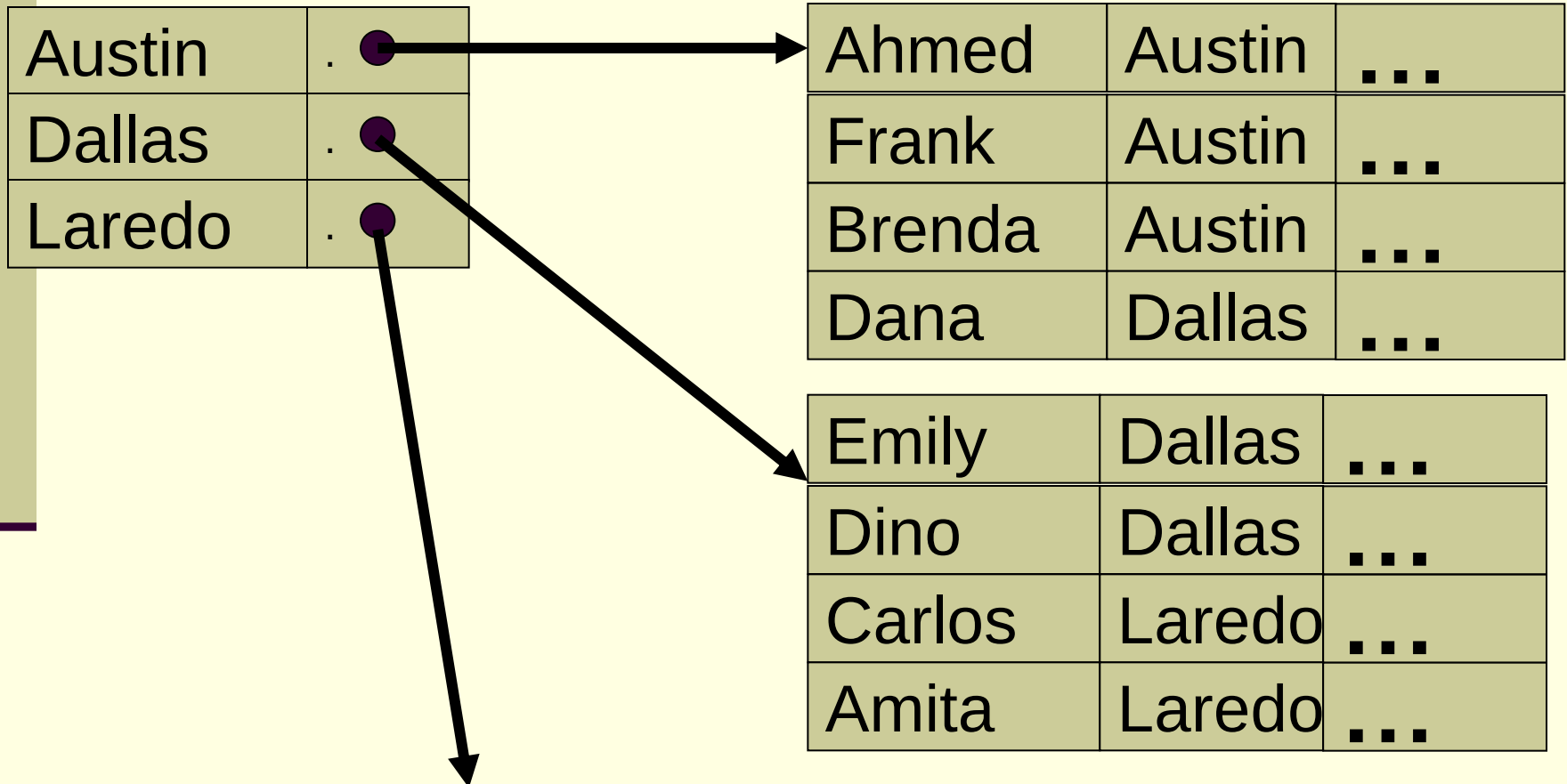


# Indexes based on other fields

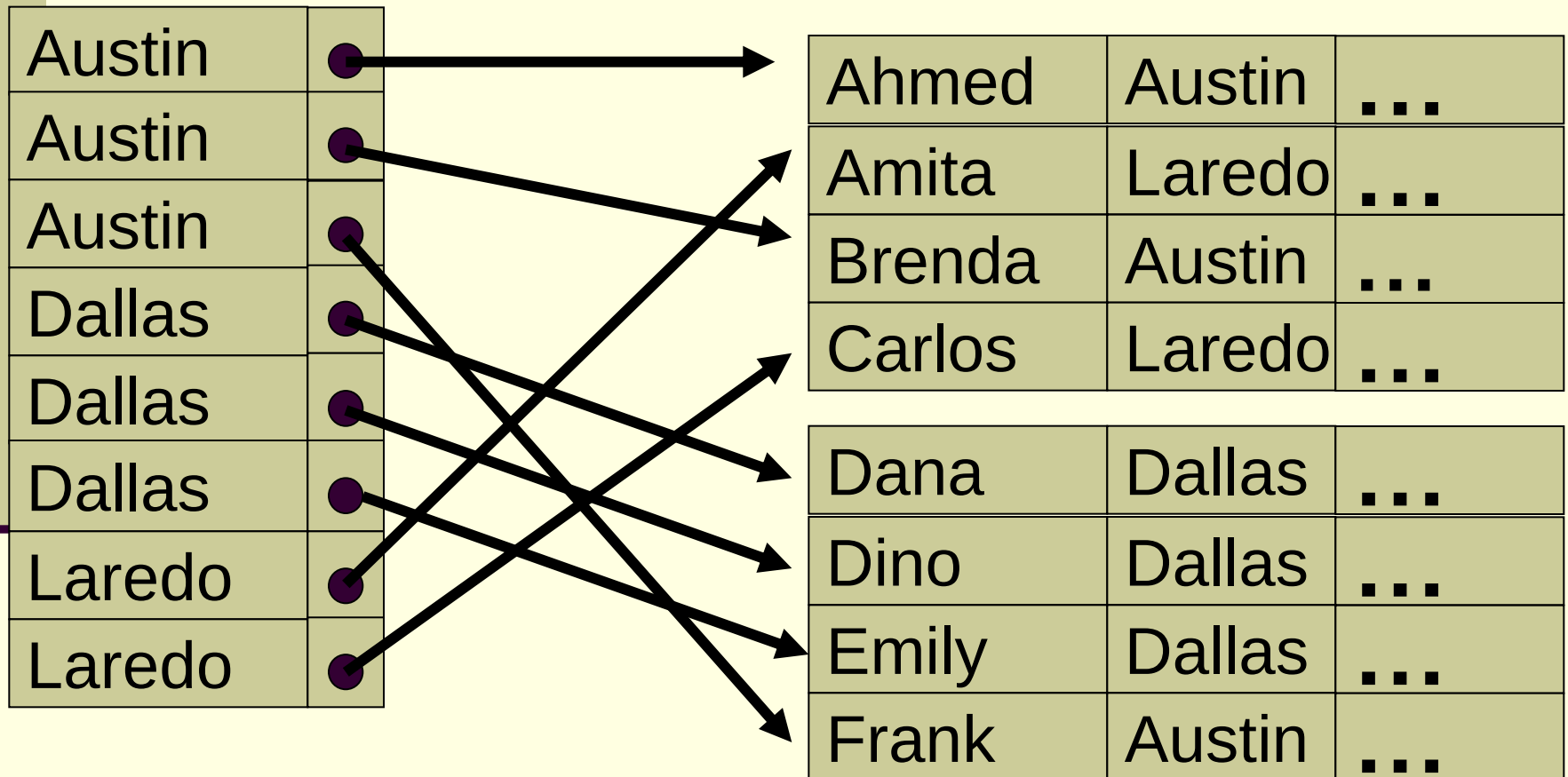
---

- Each key value may correspond to more than one record
  - ***clustering index***
- Two cases to consider:
  - Table is sorted on the field
    - Can use a sparse index
  - Table is either non-sorted or sorted on another field
    - Must use a dense index

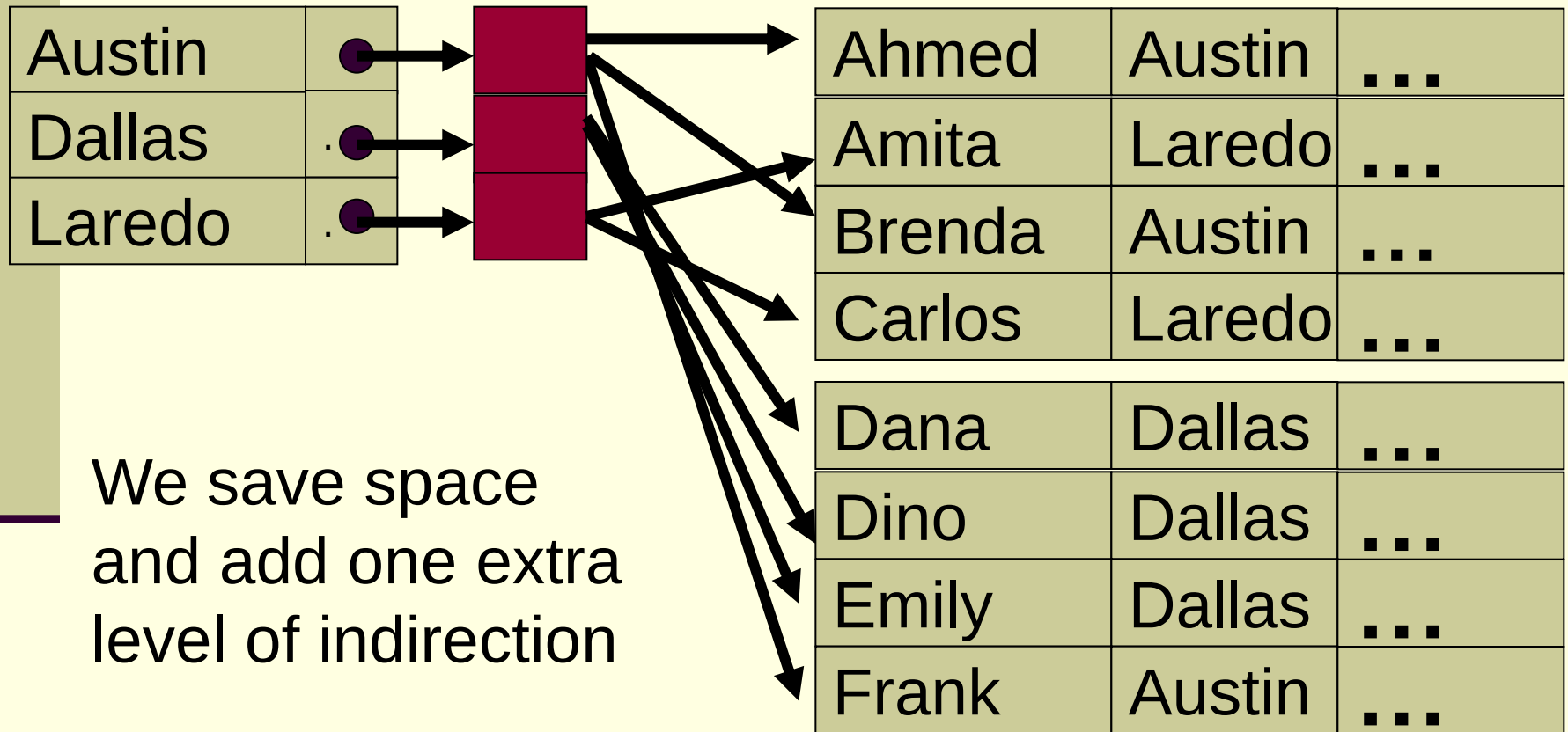
# Sparse clustering index



# Dense clustering index



# Another realization

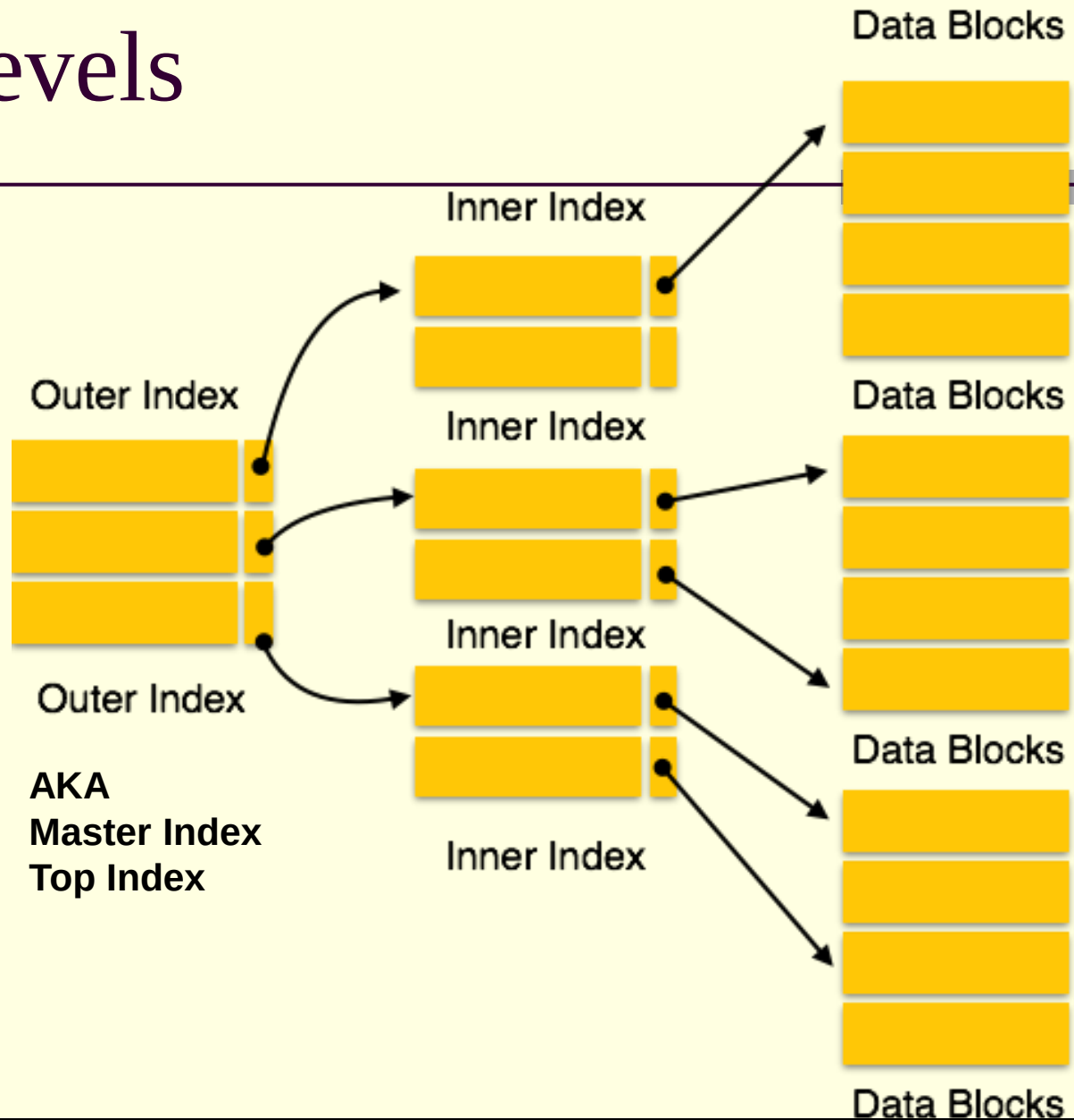


# Indexing the index

---

- When index is very large, it makes sense to index the index
  - Two-level or three-level index
  - Index at top level is called ***master index***
    - Normally a ***sparse index***

# Two levels

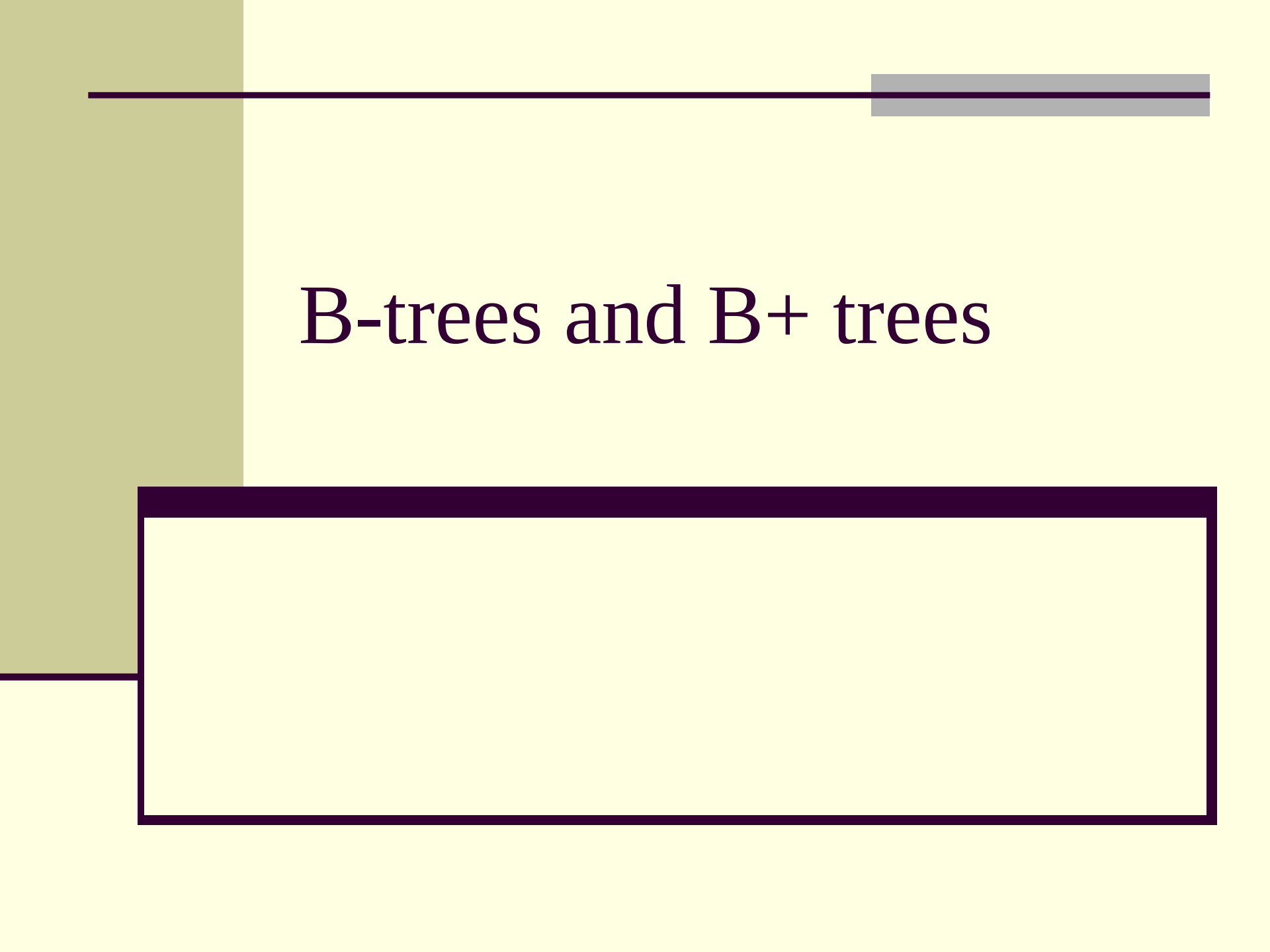


# Updating indexed tables

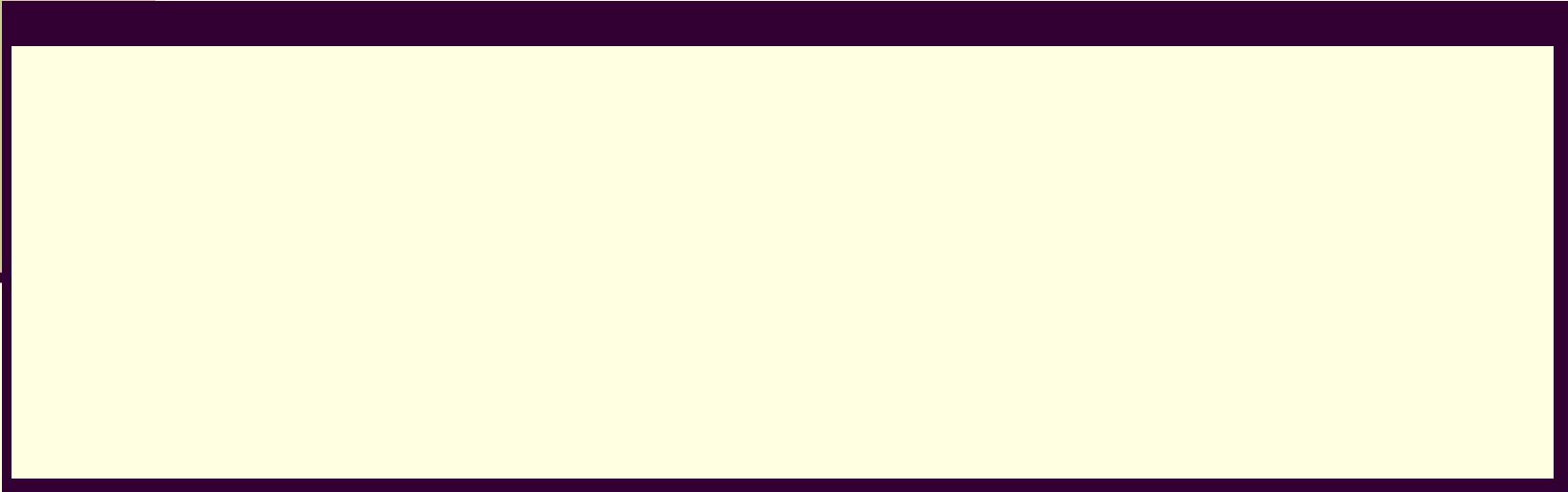
---

- Can be painful
  - No silver bullet





# B-trees and B+ trees



# Motivation

---

- To have dynamic indexing structures that can evolve when records are added and deleted
  - Not the case for static indexes
    - Would have to be completely rebuilt
- Optimized for searches on ***block devices***
- Both B trees and B+ trees are not binary
  - Objective is to increase ***branching factor*** (***degree*** or fan-out) to reduce the number of device accesses

# Binary vs. higher-order tree

---

## ■ ***Binary trees:***

- Designed for in-memory searches
- Try to minimize the number of memory accesses

## ■ ***Higher-order trees:***

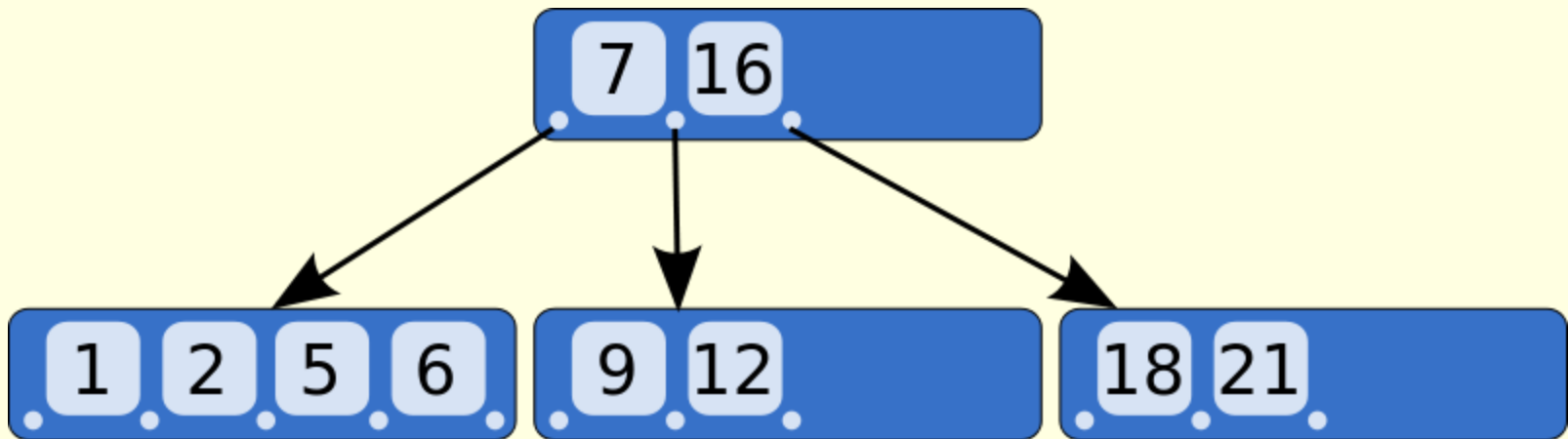
- Designed for searching data on block devices
- Try to minimize the number of device accesses
  - Searching within a block is cheap!

# B trees

---

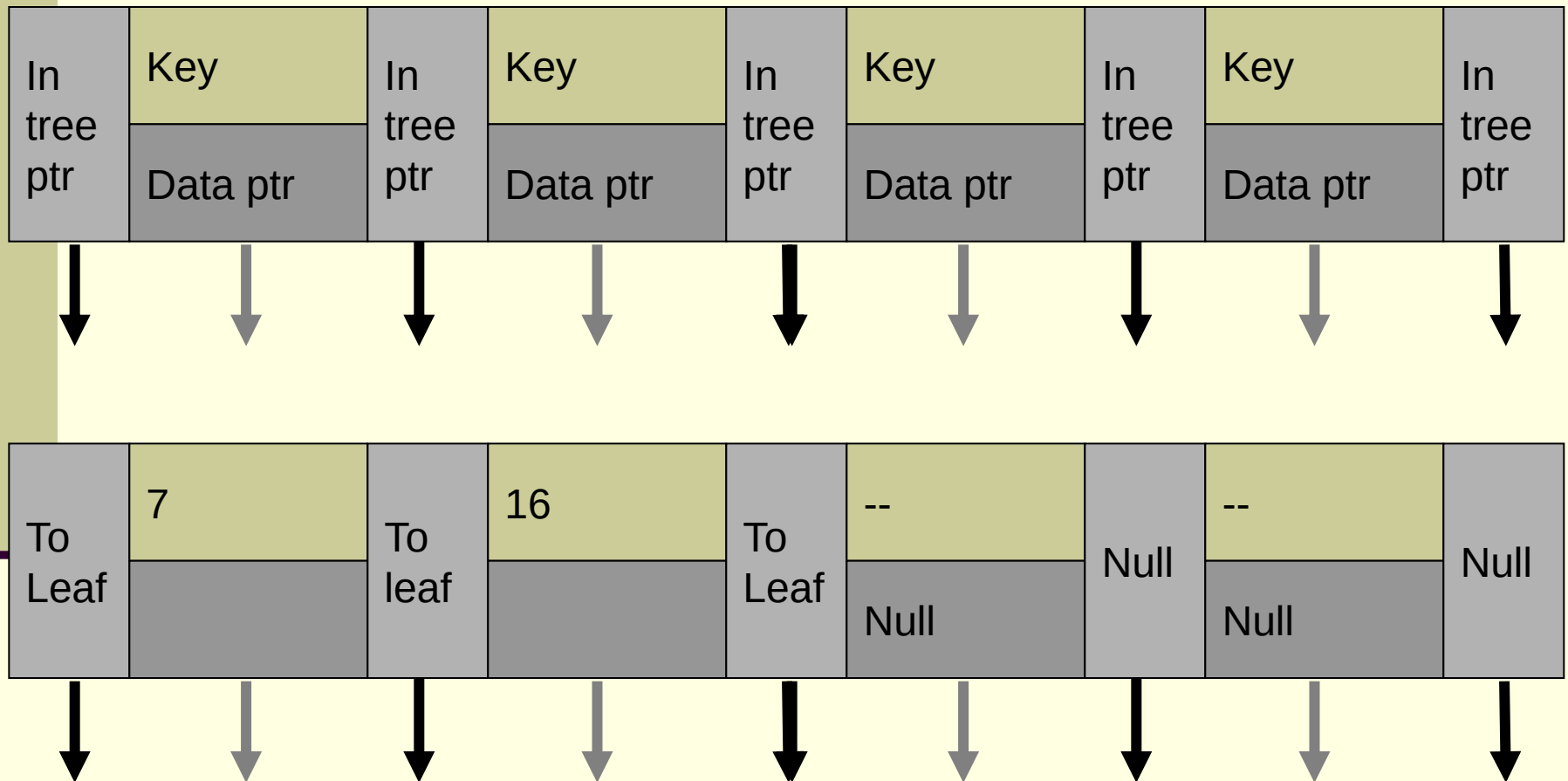
- Generalization of binary search trees
  - Not binary trees
  - The B stands for Bayer (or Boeing)
- Designed for searching data stored on block-oriented devices

# A very small B tree



Bottom nodes are leaf nodes: all their pointers are NULL

# In reality

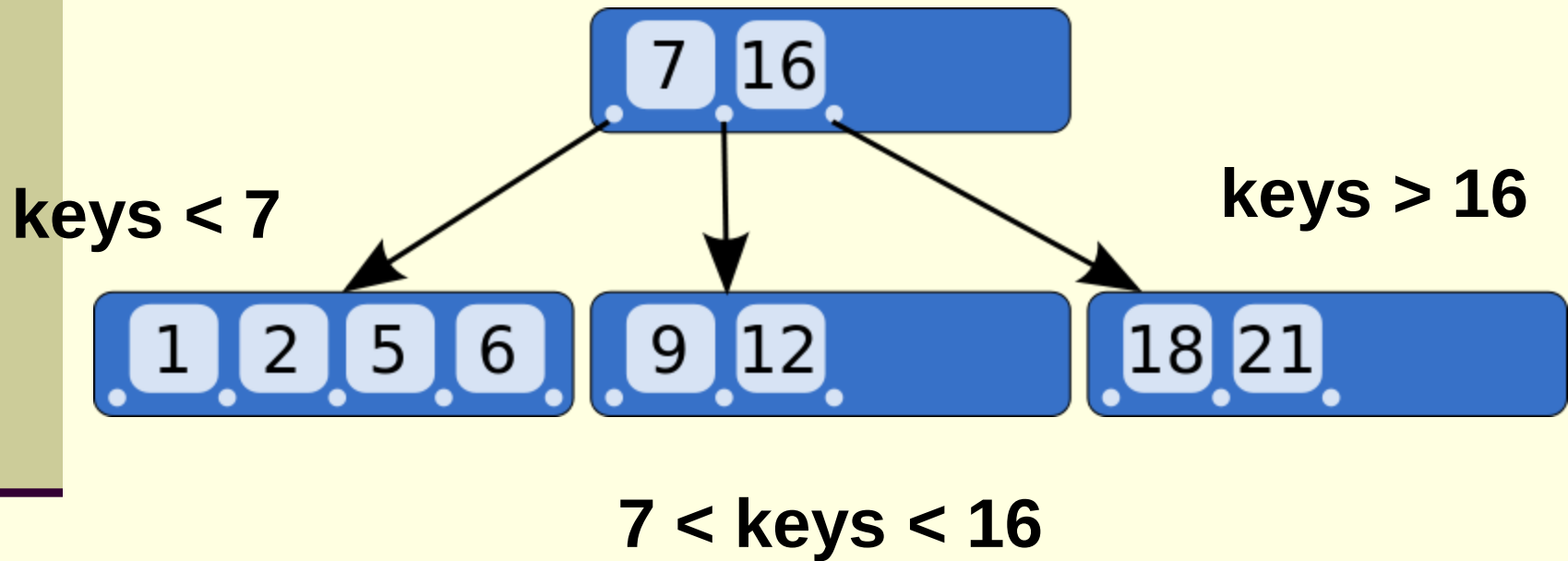


# Organization

---

- Each non-terminal node can have a variable number of child nodes
  - Must all be in a specific key range
  - Number of child nodes typically vary between  $d$  and  $2d$ 
    - Will split nodes that would otherwise have contained  $2d + 1$  child nodes
    - Will merge nodes that contain less than  $d$  child nodes

# Searching the tree





# Balancing B trees

---

- Objective is to ensure that all terminal nodes be at the same depth
  - Root node can have min two branches.
  - All leaf nodes at same level.
  - Every node should have  $(m/2)$  branches.
  - Deletion is bottom up approach.

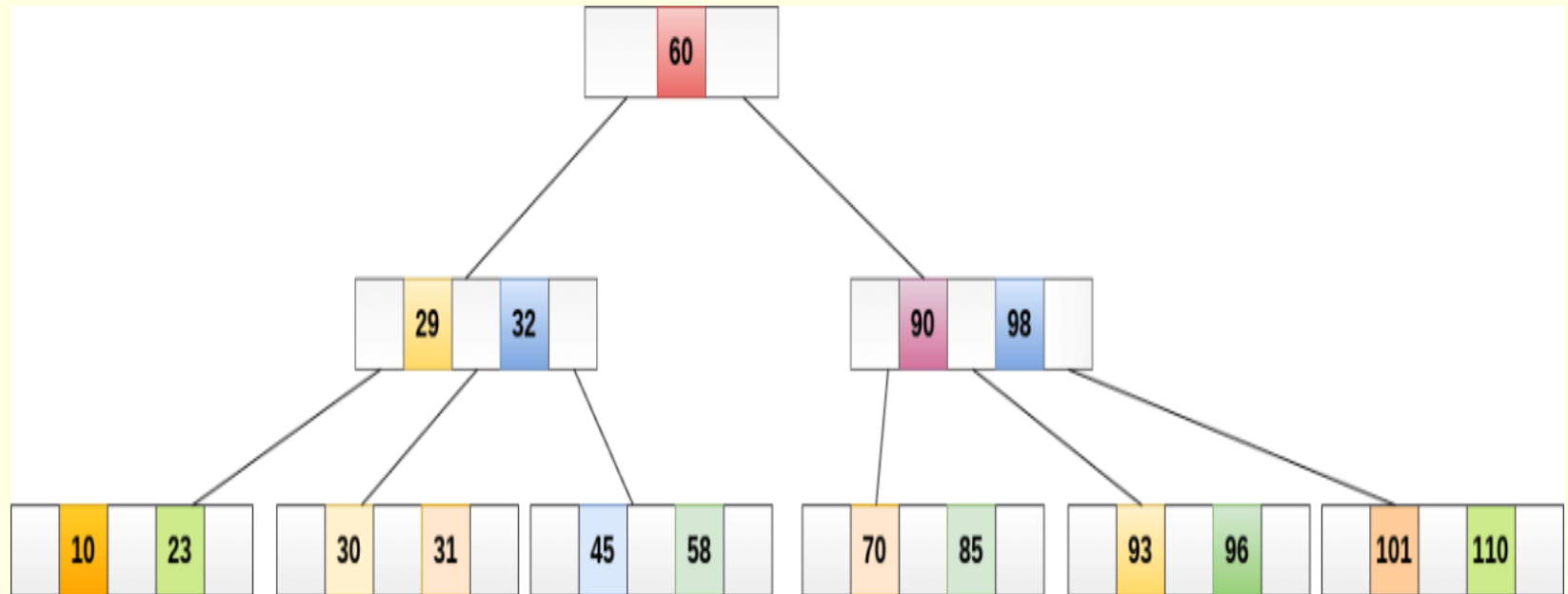
# B Tree

---

- B Tree is a specialized m-way tree that can be widely used for disk access. A B-Tree of order m can have at most m-1 keys and m children. One of the main reason of using B tree is its capability to store large number of keys in a single node and large key values by keeping the height of the tree relatively small.

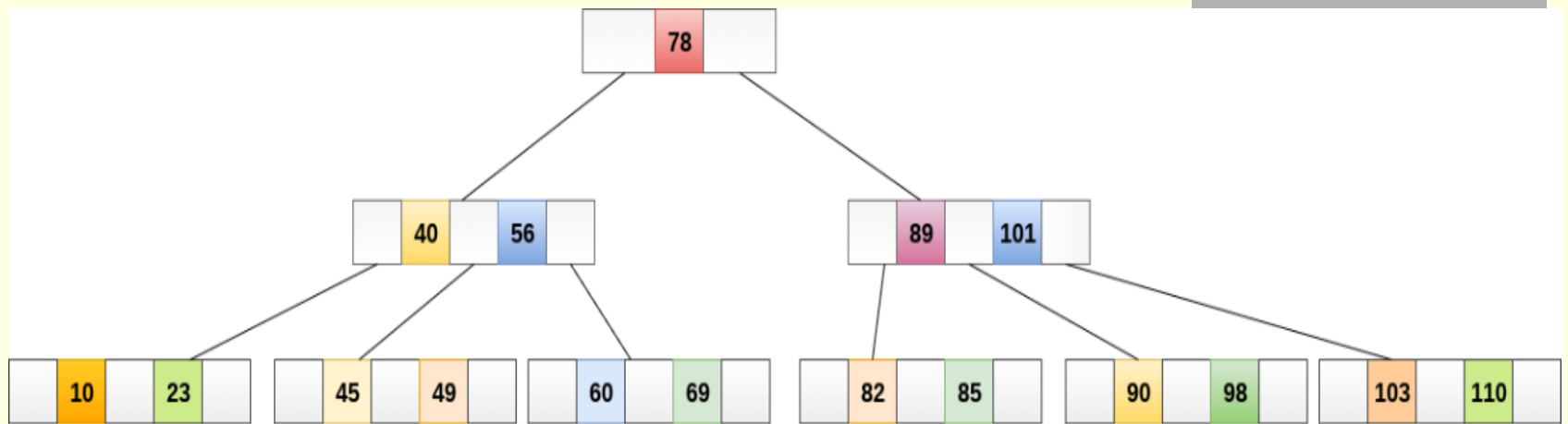
- 
- A B tree of order  $m$  contains all the properties of an  $M$  way tree. In addition, it contains the following properties.
  - Every node in a B-Tree contains at most  $m$  children.
  - Every node in a B-Tree except the root node and the leaf node contain at least  $m/2$  children.
  - The root nodes must have at least 2 nodes.
  - All leaf nodes must be at the same level.
  - It is not necessary that, all the nodes contain the same number of children but, each node must have  $m/2$  number of nodes.

# B Tree of order 4



# Searching :

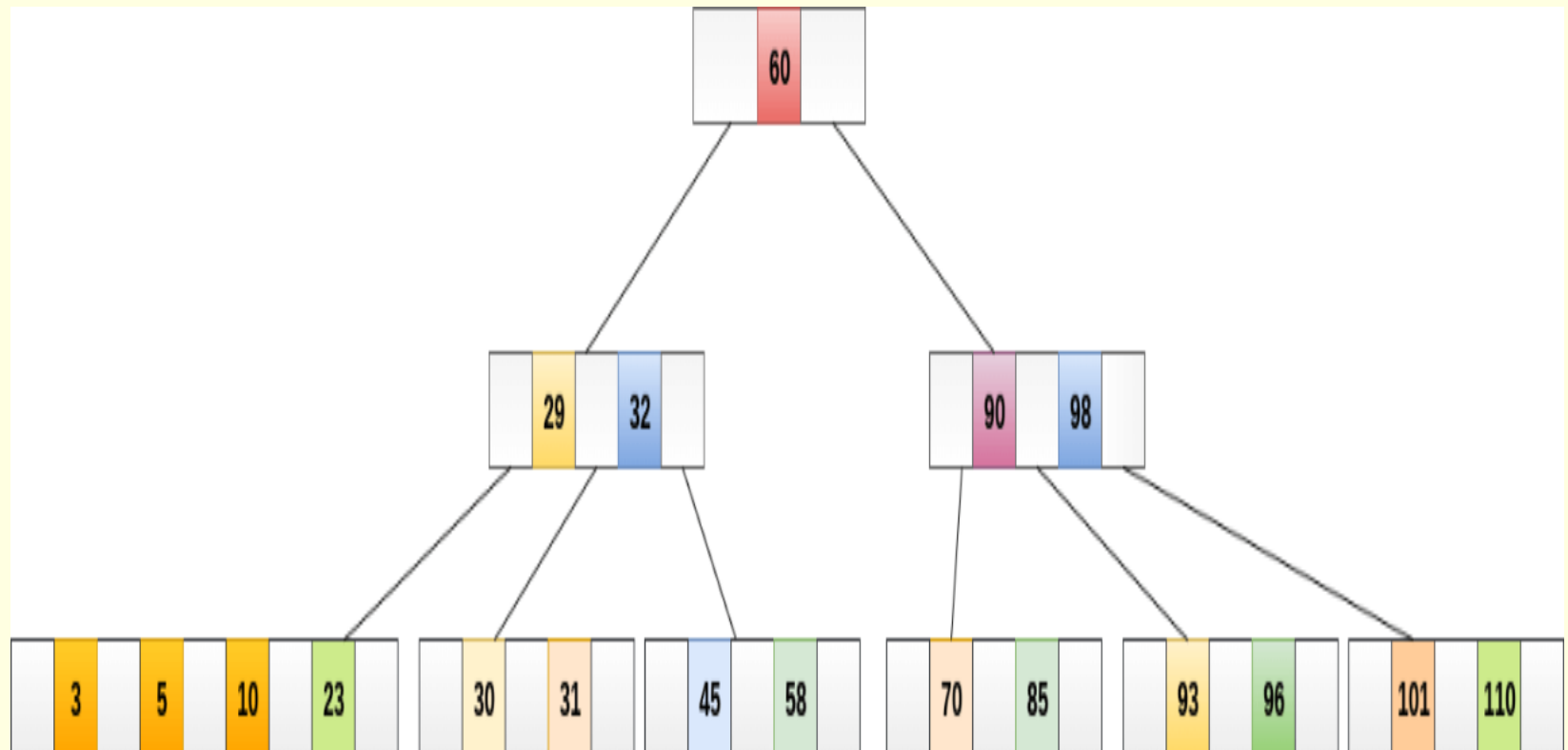
- Searching in B Trees is similar to that in Binary search tree. For example, if we search for an item 49 in the following B Tree. The process will something like following :
- Compare item 49 with root node 78. since  $49 < 78$  hence, move to its left sub-tree.
- Since,  $40 < 49 < 56$ , traverse right sub-tree of 40.
- $49 > 45$ , move to right. Compare 49.
- match found, return.
- Searching in a B tree depends upon the height of the tree. The search algorithm takes  $O(\log n)$  time to search any element in a B tree.



# Insertion

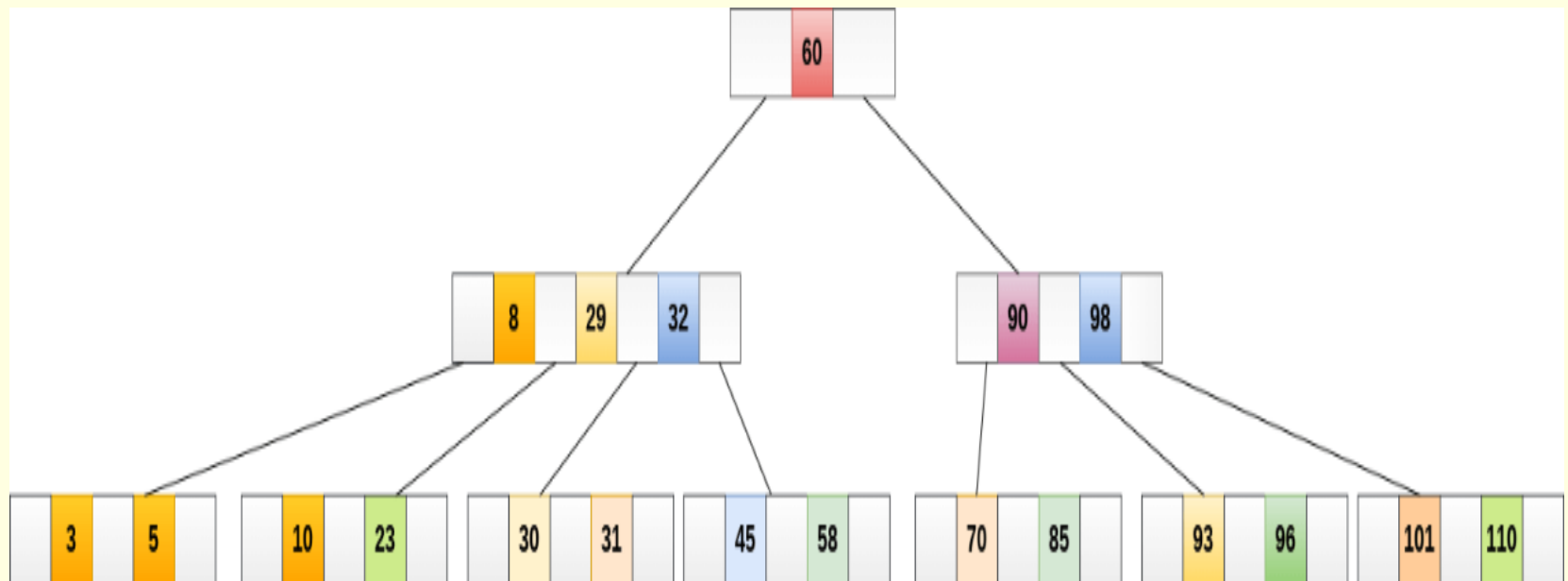
- Insertions are done at the leaf node level. The following algorithm needs to be followed in order to insert an item into B Tree.
- Traverse the B Tree in order to find the appropriate leaf node at which the node can be inserted.
- If the leaf node contain less than  $m-1$  keys then insert the element in the increasing order.
- Else, if the leaf node contains  $m-1$  keys, then follow the following steps.
  - Insert the new element in the increasing order of elements.
  - Split the node into the two nodes at the median.
  - Push the median element upto its parent node.
  - If the parent node also contain  $m-1$  number of keys, then split it too by following the same steps.

# Given Tree





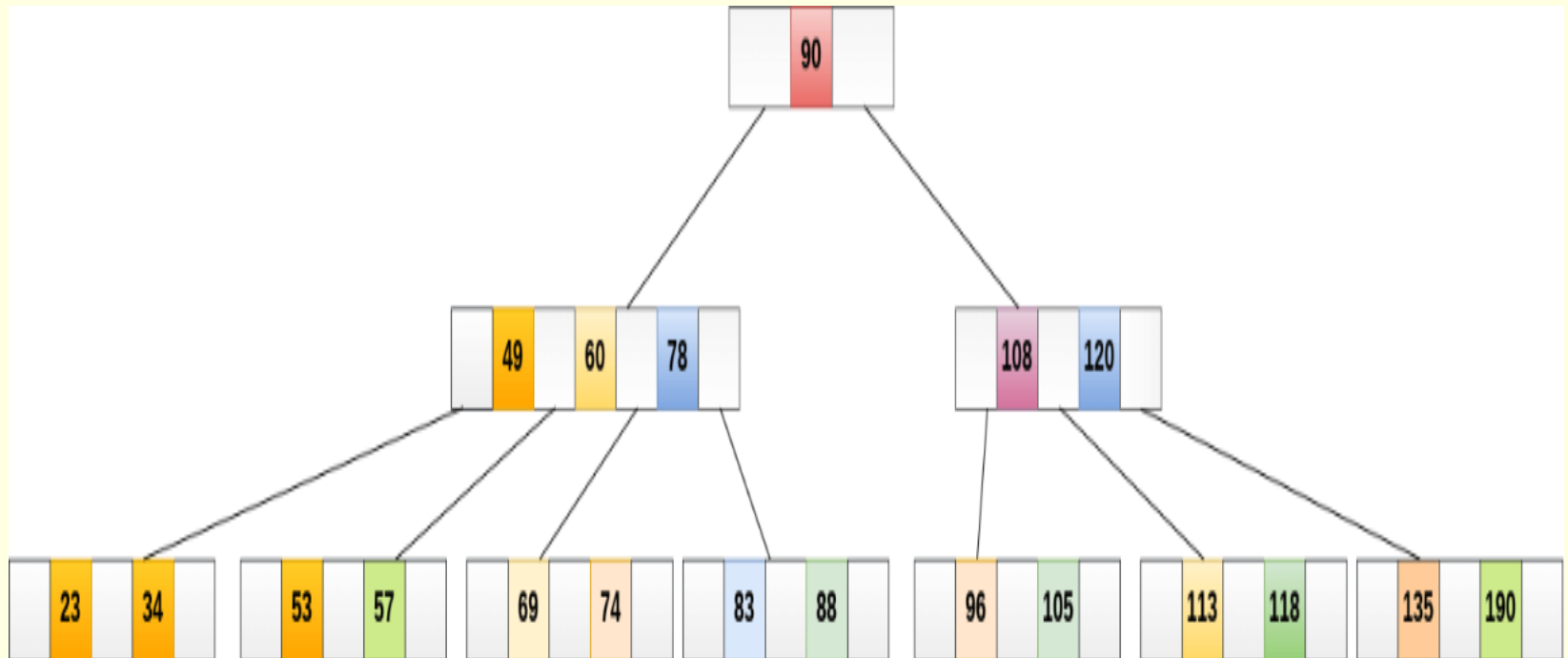
# Insert 8



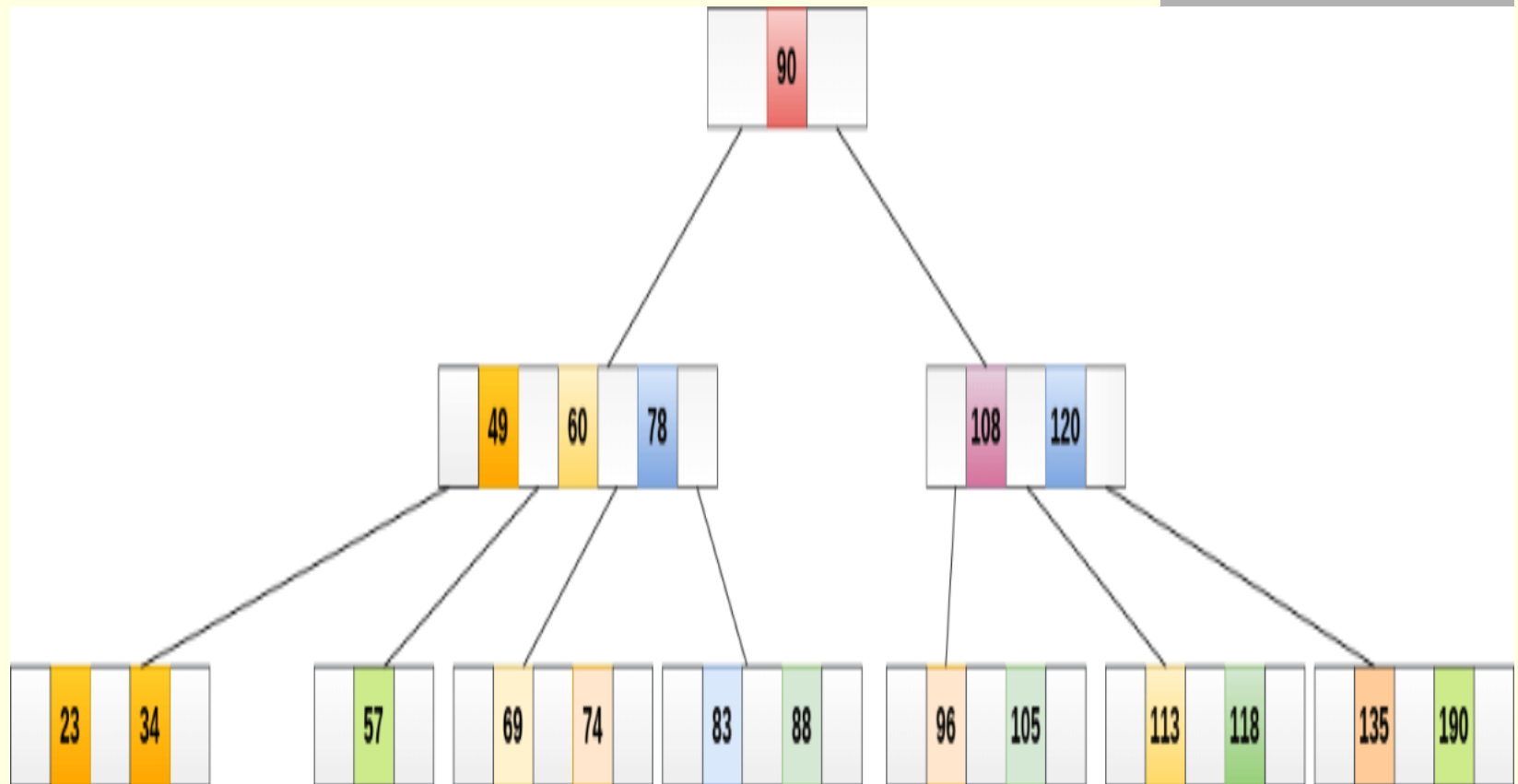
- Deletion is also performed at the leaf nodes. The node which is to be deleted can either be a leaf node or an internal node.
- Locate the leaf node.
- If there are more than  $m/2$  keys in the leaf node then delete the desired key from the node.
- If the leaf node doesn't contain  $m/2$  keys then complete the keys by taking the element from right or left sibling.
  - If the left sibling contains more than  $m/2$  elements then push its largest element up to its parent and move the intervening element down to the node where the key is deleted.
  - If the right sibling contains more than  $m/2$  elements then push its smallest element up to the parent and move intervening element down to the node where the key is deleted.

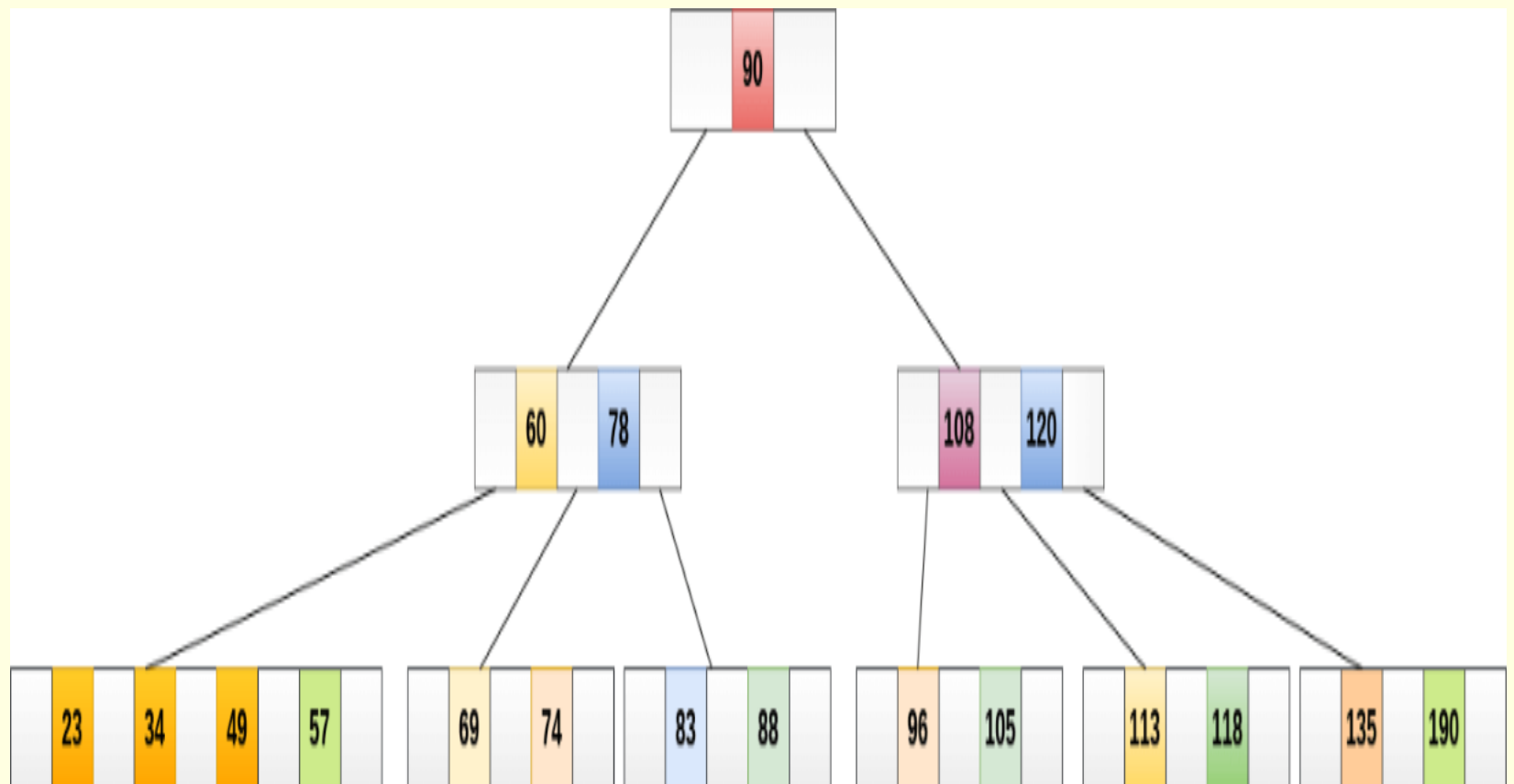
- If neither of the sibling contain more than  $m/2$  elements then create a new leaf node by joining two leaf nodes and the intervening element of the parent node.
- If parent is left with less than  $m/2$  nodes then, apply the above process on the parent too.
- If the the node which is to be deleted is an internal node, then replace the node with its in-order successor or predecessor. Since, successor or predecessor will always be on the leaf node hence, the process will be similar as the node is being deleted from the leaf node.

# B Tree



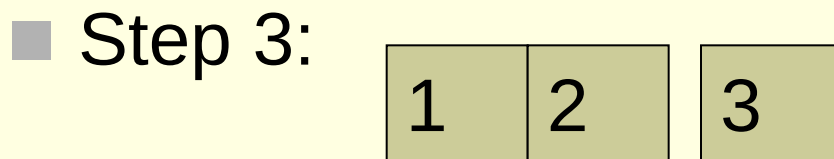
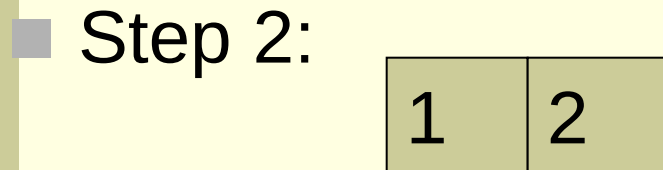
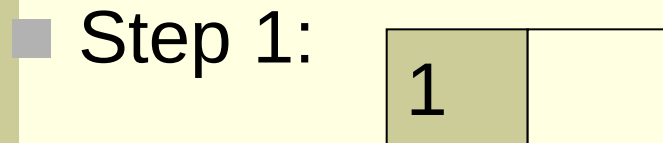
# Delete 53



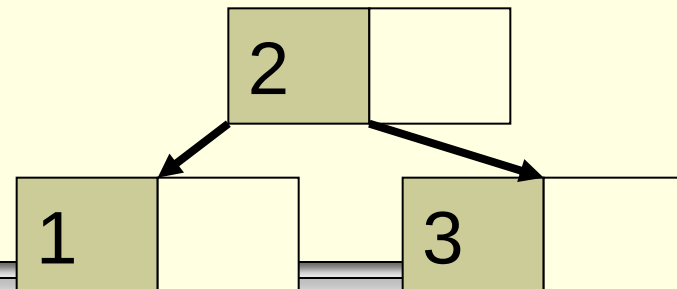


# Insertions

- Assume a tree where each node can contain three pointers (non represented)

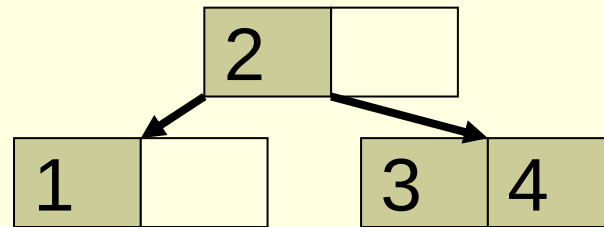


■ Split node in middle



# Insertions

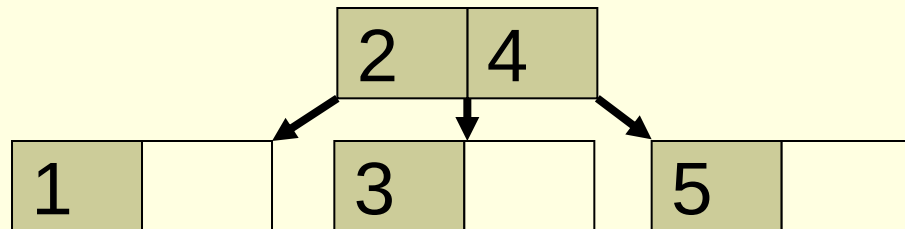
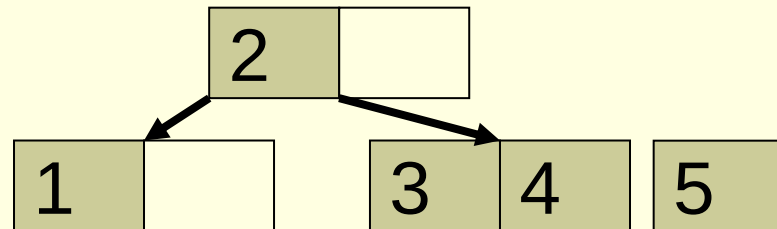
■ Step 4:



■ Step 5:

■ Split

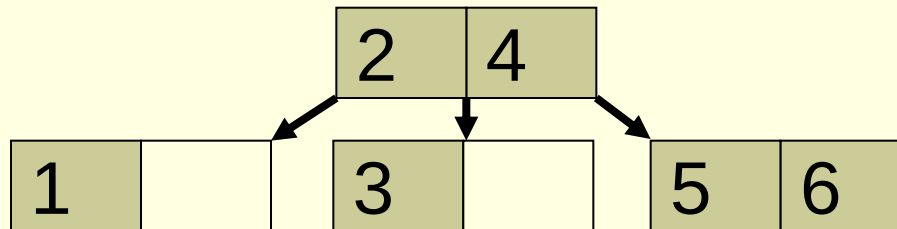
■ Move up



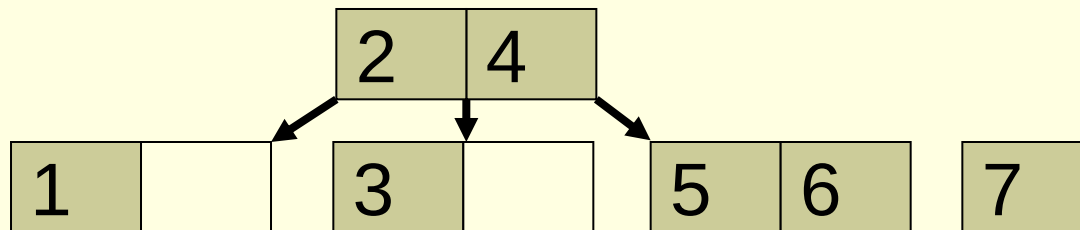


# Insertions

## ■ Step 6:

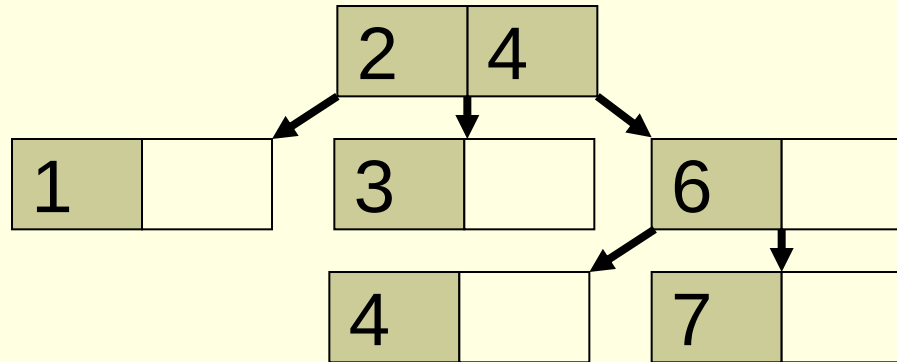


## ■ Step 7:

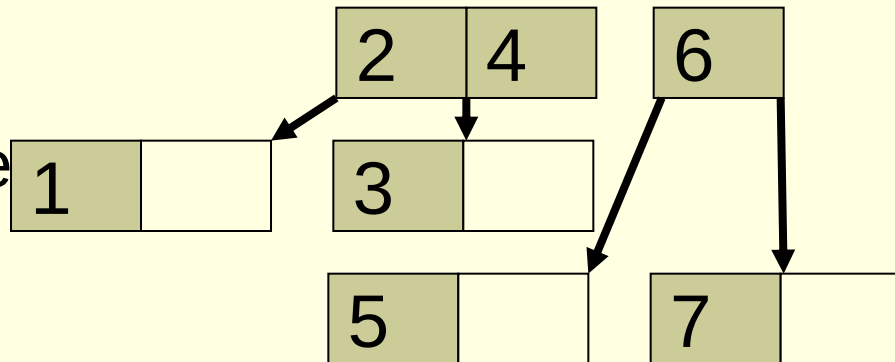


# Step 7 continued

■ Split

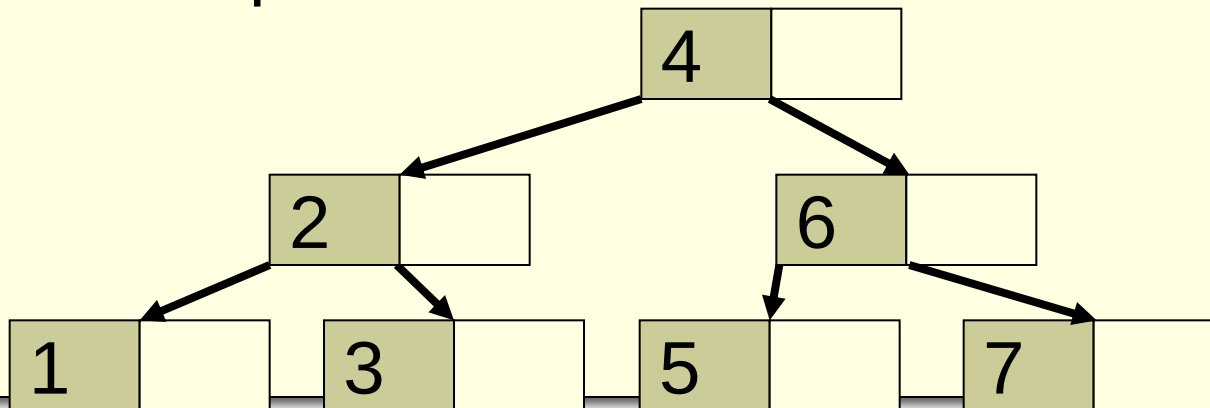
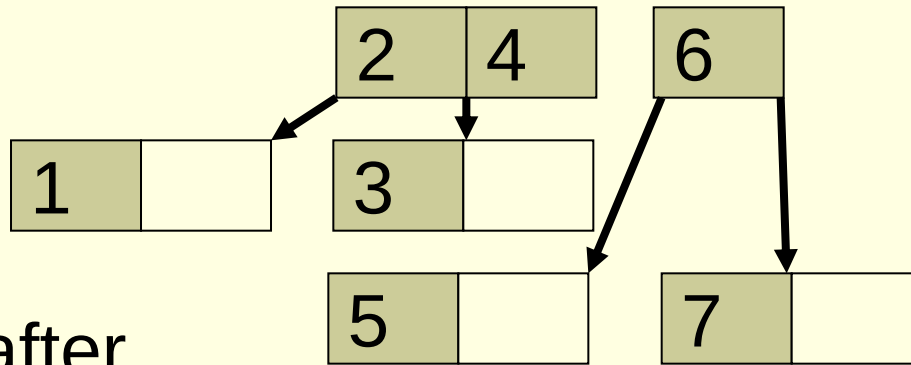


■ Promote



# Step 7 continued

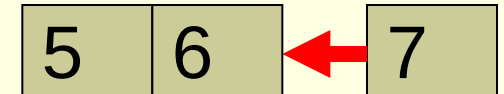
- Split after the promotion



# Two basic operations

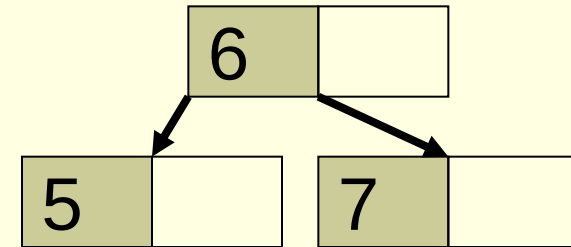
## ■ **Split:**

- When trying to add to a full node
- Split node at central value



## ■ **Promote:**

- Must insert root of split node higher up
- May require a new split

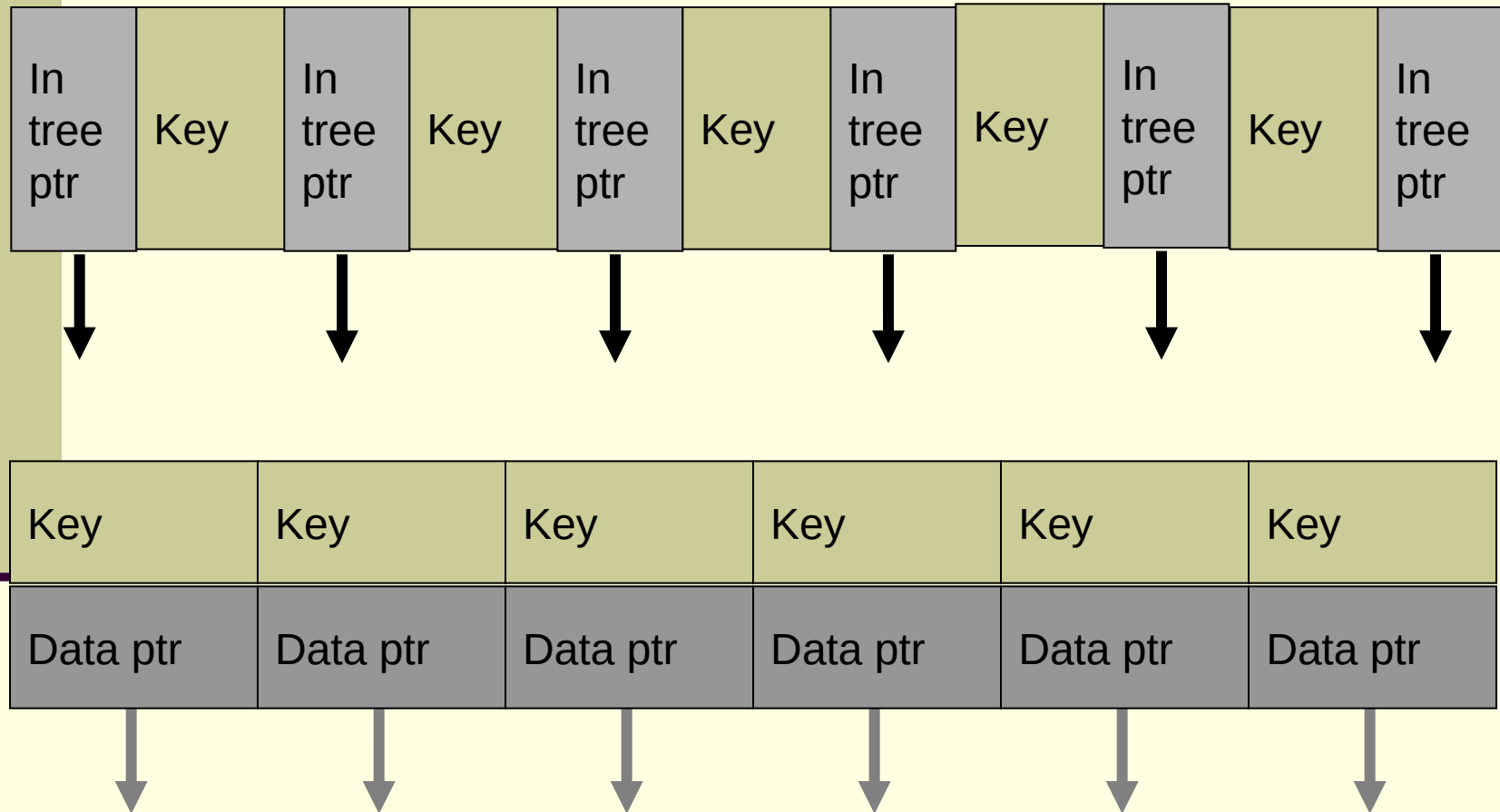


# B+ trees

---

- Variant of B trees
- Two types of nodes
  - Internal nodes have no data pointers
  - Leaf nodes have no in-tree pointers
    - Were all null!

# B+ tree nodes



# More about internal nodes

- Consist of  $n - 1$  key values  $K_1, K_2, \dots, K_{n-1}$ , and  $n$  tree pointers  $P_1, P_2, \dots, P_n$ :
  - $\langle P_1, K_1, P_2, K_2, P_3, \dots, P_{n-1}, K_{n-1}, P_n \rangle$
- The keys are ordered  $K_1 < K_2 < \dots < K_{n-1}$
- For each tree value  $X$  in the subtree pointed at by tree pointer  $P_i$ , we have:
  - $X > K_{i-1}$  for  $1 \leq i \leq n$
  - $X \leq K_i$  for  $1 \leq i \leq n - 1$

# Warning

- Other authors assume that
  - For each tree value  $X$  in the subtree pointed at by tree pointer  $P_i$ , we have:
    - $X \geq K_{i-1}$  for  $1 \leq i \leq n$
    - $X < K_i$  for  $1 \leq i \leq n - 1$
- Changes the key value that is promoted when an internal node is split



# Advantages

---

- Removing unneeded pointers allows to pack more keys in each node
  - Higher fan-out for a given node size
    - Normally one block
- Having all keys present in the leaf nodes allows us to build a linked list of all keys

# Properties

---

- If  $m$  is the order of the tree
  - Every internal node has at most  $m$  children.
  - Every internal node (except root) has at least  $\lceil m/2 \rceil$  children.
  - The root has at least two children if it is not a leaf node.
  - Every leaf has at most  $m - 1$  keys
  - An internal node with  $k$  children has  $k - 1$  keys.
  - All leaves appear in the same level

# Best cases and worst cases

---

- A B+ tree of degree  $m$  and height  $h$  will store
  - At most  $m^{h-1}(m-1) = \mathbf{m^h - m}$  records
  - At least  $\mathbf{2\lceil m/2 \rceil^{h-1}}$  records

# Insertions

- **def** insert (entry) :
  - Find target leaf  $L$
  - **if**  $L$  has less than  $m - 2$  entries :
    - add the entry
  - else** :
    - Allocate new leaf  $L'$
    - Pick the  $m/2$  highest keys of  $L$  and move them to  $L'$
    - Insert **highest key** of  $L$  and corresponding address leaf into the parent node
    - If the parent is full :
      - Split it and add the middle key to its parent node
    - Repeat until a parent is found that is not full

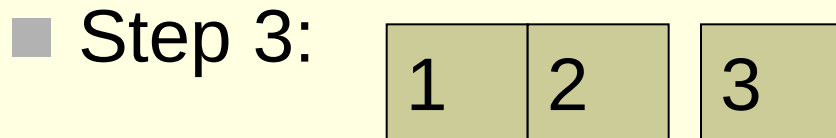
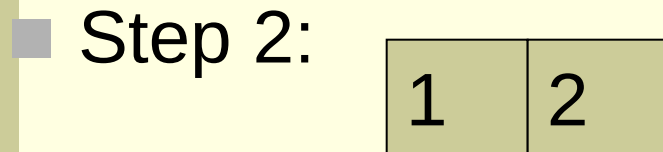
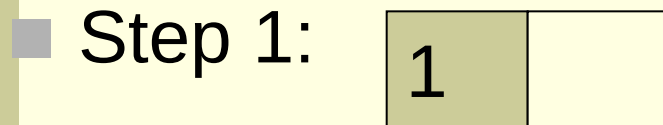
# Deletions

---

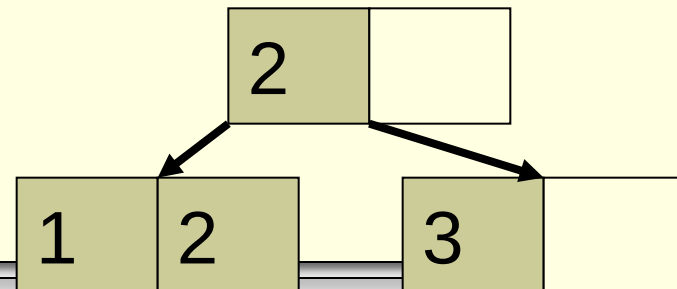
- **def** delete (record) :
  - Locate target leaf and remove the entry
  - If leaf is less than half full:
    - Try to re-distribute, taking from sibling (adjacent node with same parent)
    - If re-distribution fails:
      - Merge leaf and sibling
      - Delete entry to one of the two merged leaves
      - Merge could propagate to root

# Insertions

- Assume a B+ tree of degree 3

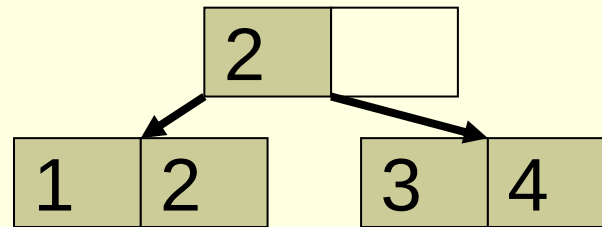


- Split node in middle



# Insertions

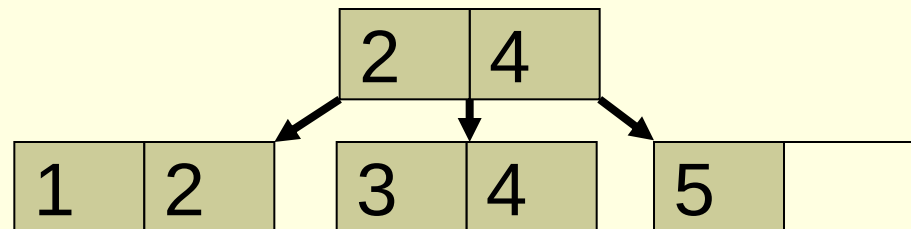
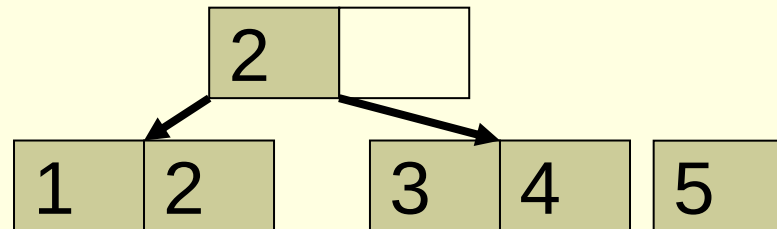
■ Step 4:



■ Step 5:

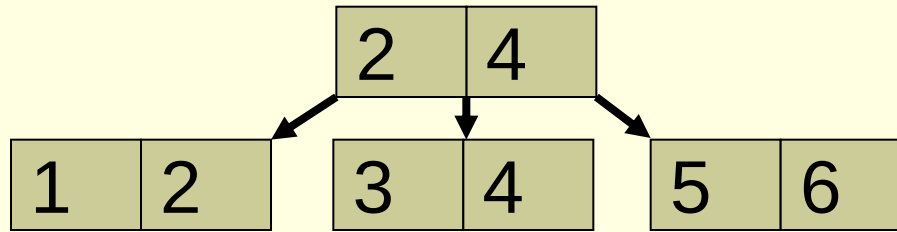
■ Split

■ Move up

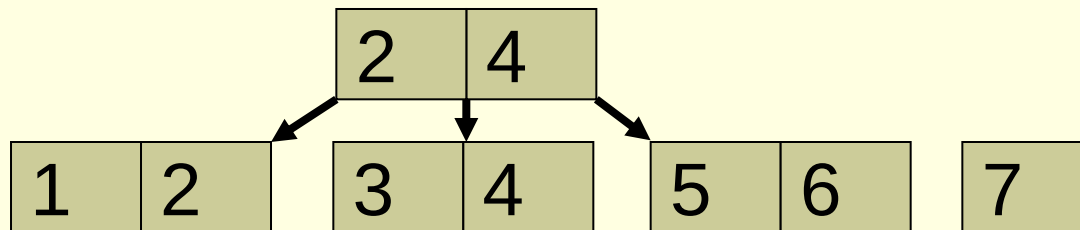


# Insertions

## ■ Step 6:

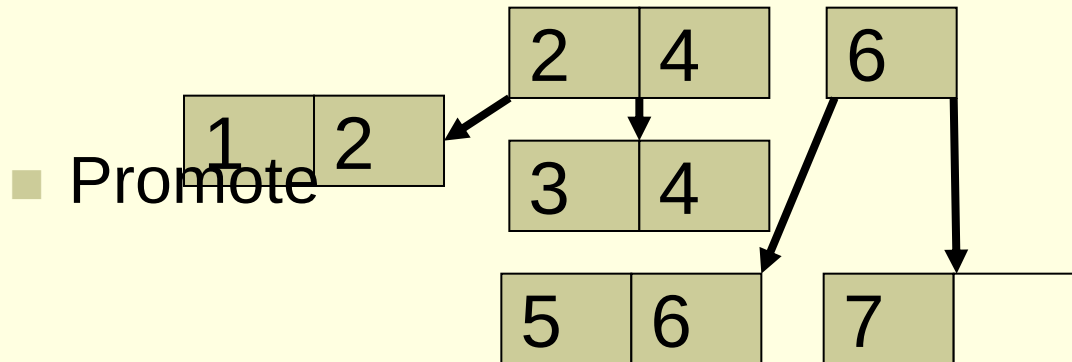
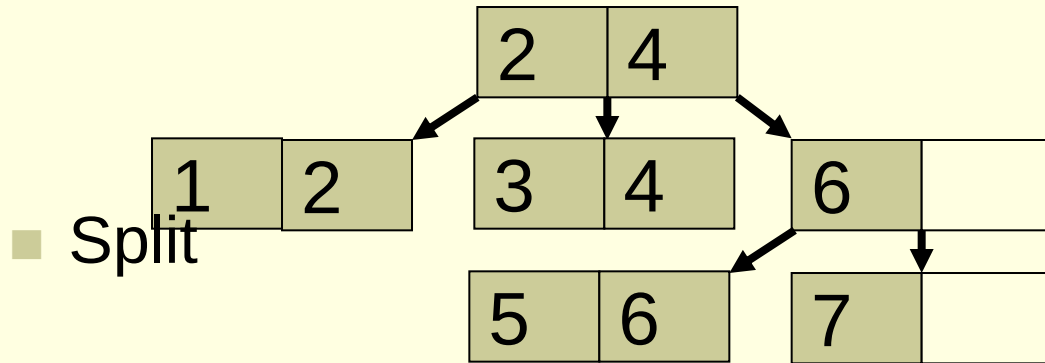


## ■ Step 7:



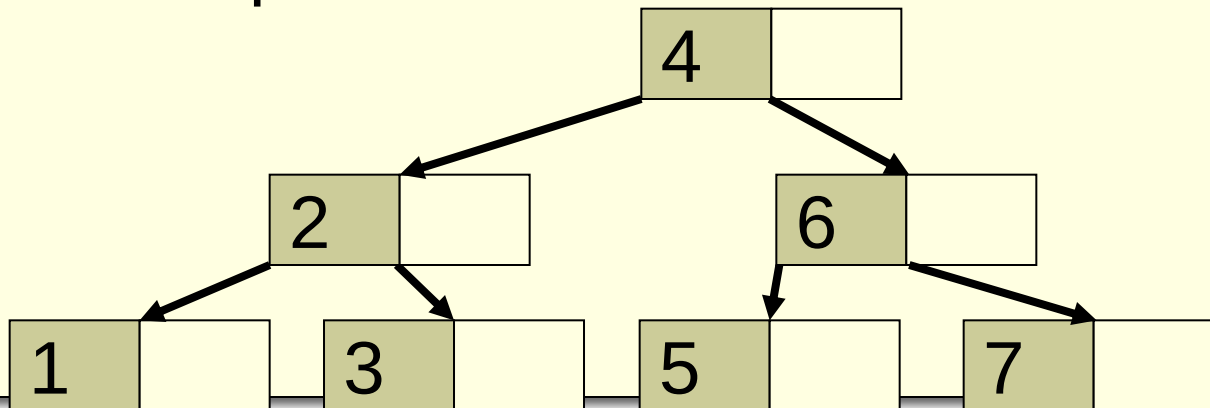
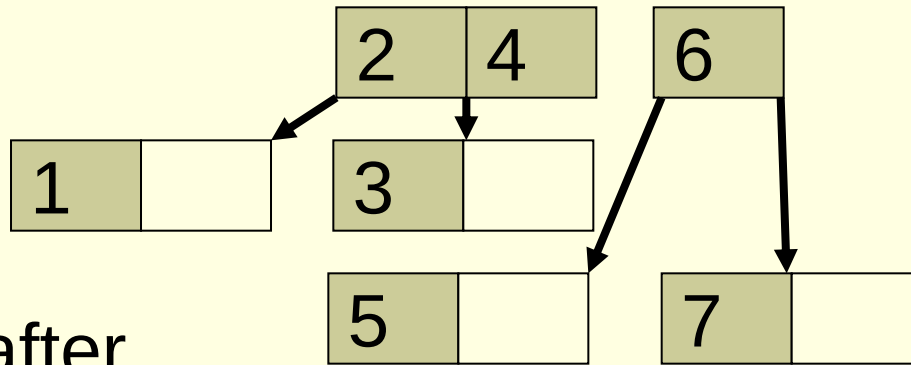


# Step 7 continued



# Step 7 continued

- Split after the promotion



# Importance

---

- B+ trees are used by
  - NTFS, ReiserFS, NSS, XFS, JFS, ReFS, and BFS file systems for metadata indexing
  - BFS for storing directories.
  - IBM DB2, Informix, Microsoft SQL Server, Oracle 8, Sybase ASE, and SQLite for table indexes

# An interesting variant

---

- Can simplify entry deletion by never merging nodes that have less than  $\lceil m/2 \rceil$  entries
- Wait instead until there are empty and can be deleted
- Requires more space
- Seems to be a reasonable tradeoff assuming random insertions and deletions

# B+ Trees

---

- What are B+ Trees used for
- What is a B Tree
- What is a B+ Tree
- Searching
- Insertion
- Deletion

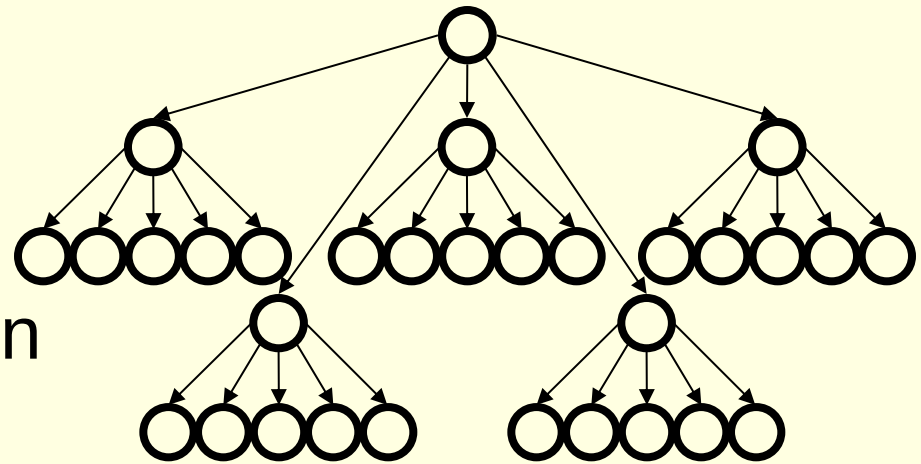
# What are B+ Trees Used For?

---

- When we store data in a table in a DBMS we want
  - Fast lookup by primary key
    - Just this – hashtable  $O(c)$
  - Ability to add/remove records on the fly
    - Some kind of dynamic tree **on disk**
  - Sequential access to records (physically sorted by primary key on disk)
    - Tree structured keys (hierarchical index for searching)
    - Records all at leaves in sorted order

# What is an M-ary Search Tree?

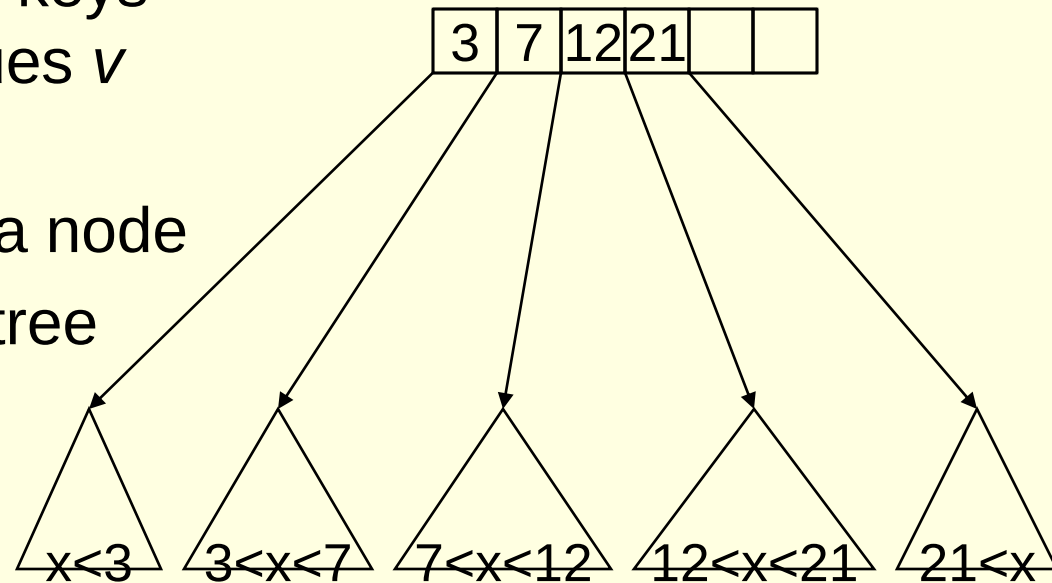
- Maximum branching factor of  $M$
- Complete tree has depth  $= \lceil \log_M N \rceil$
- Each internal node in a complete tree has  $M - 1$  keys



Binary search tree is a  
B Tree where  $M$  is 2

# B Trees

- B-Trees are specialized  $M$ -ary search trees
- Each node has many keys
  - subtree between two keys  $x$  and  $y$  contains values  $v$  such that  $x \leq v < y$
  - binary search within a node
  - to find correct subtree
- Each node takes one
  - full  $\{page, block, line\}$
  - of memory (disk)





# B-Tree Properties

---

- Properties

- maximum branching factor of  $M$
- the root has between 2 and  $M$  children *or* at most  $M-1$  keys
- All other nodes have between  $\lceil M/2 \rceil$  and  $M$  records
  - Keys+data

- Result

- tree is  $O(\log M)$  deep
- all operations run in  $O(\log M)$  time
- operations pull in about  $M$  items at a time

# What is a B+ Tree?

---

- A variation of B trees in which
  - internal nodes contain only search keys (no data)
  - Leaf nodes contain pointers to data records
  - Data records are in sorted order by the search key
  - All leaves are at the same depth

# Definition of a B+Tree

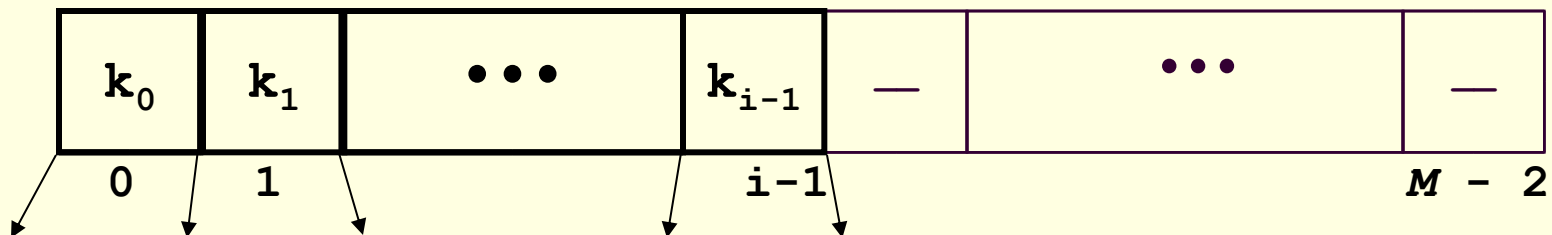
---

A B+ tree is a balanced tree in which every path from the root of the tree to a leaf is of the same length, and each non-leaf node of the tree has between  $\lceil M/2 \rceil$  and  $\lceil M \rceil$  children, where  $n$  is fixed for a particular tree.

# B+ Tree Nodes

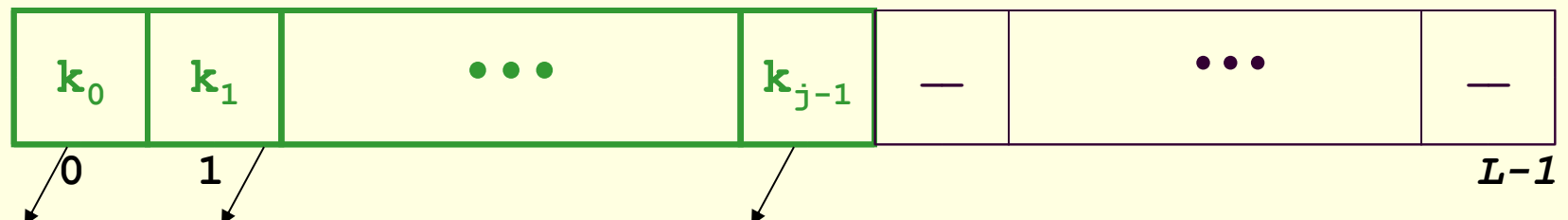
## Internal node

- Pointer (Key, NodePointer)\*M-1 in each node
- First  $i$  keys are currently in use



## Leaf

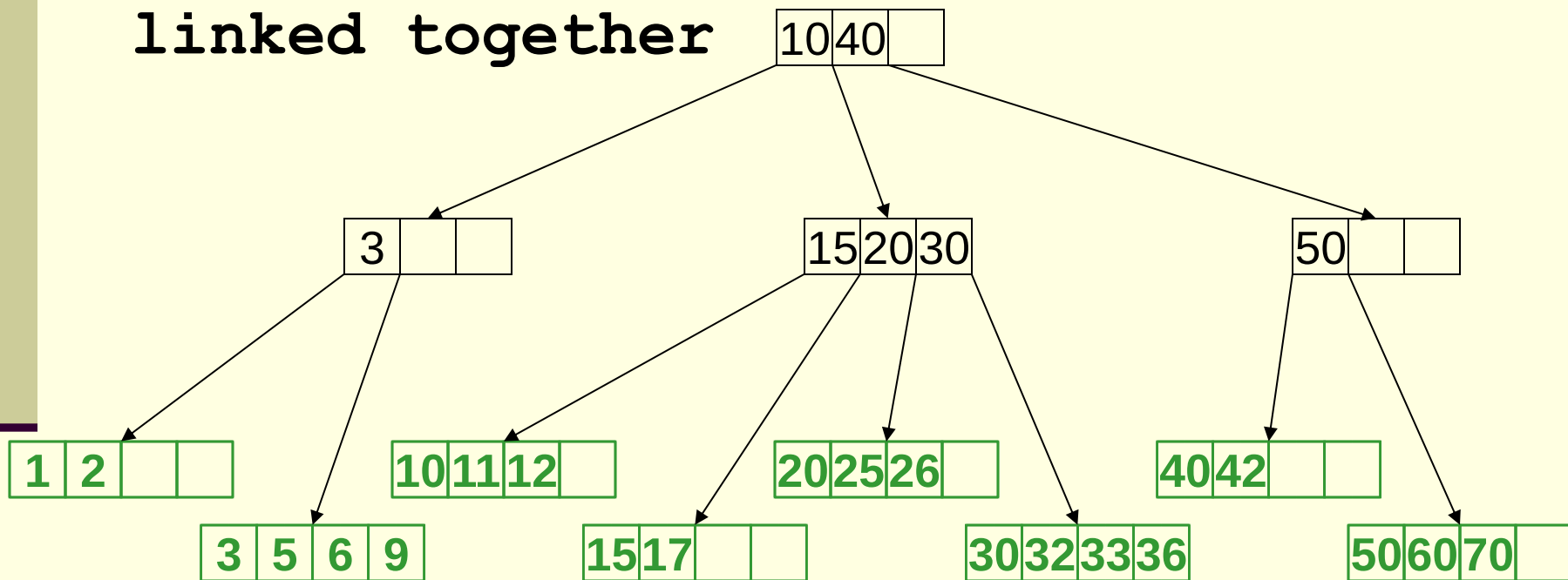
- (Key, DataPointer)\* L in each node
- first  $j$  Keys currently in use



# Example

B+ Tree with  $M = 4$

Often, leaf nodes  
linked together



# Advantages of B+ tree usage for databases

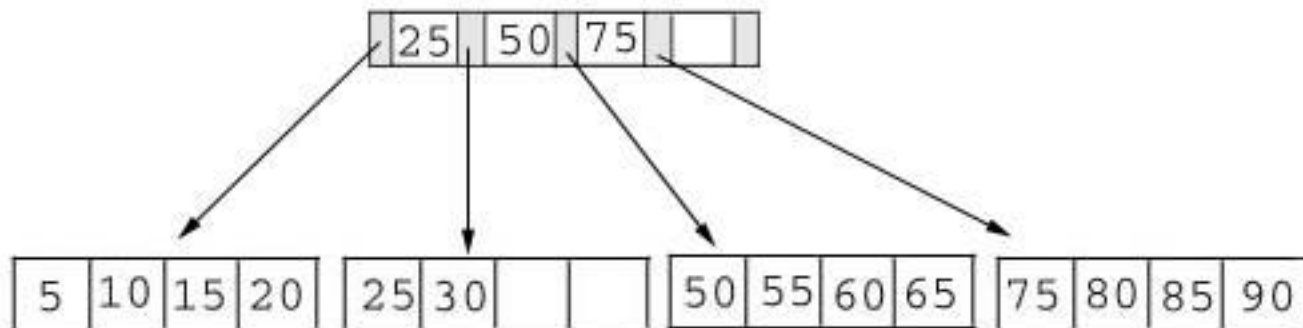
---

- keeps keys in sorted order for sequential traversing
- uses a hierarchical index to minimize the number of disk reads
- uses partially full blocks to speed insertions and deletions
- keeps the index balanced with a recursive algorithm
- In addition, a B+ tree minimizes waste by making sure the interior nodes are at least half full. A B+ tree can handle an arbitrary number of insertions and deletions.

# Searching

- Just compare the key value with the data in the tree, then return the result.

For example: find the value **45**, and **15** in below tree.



# Searching

---

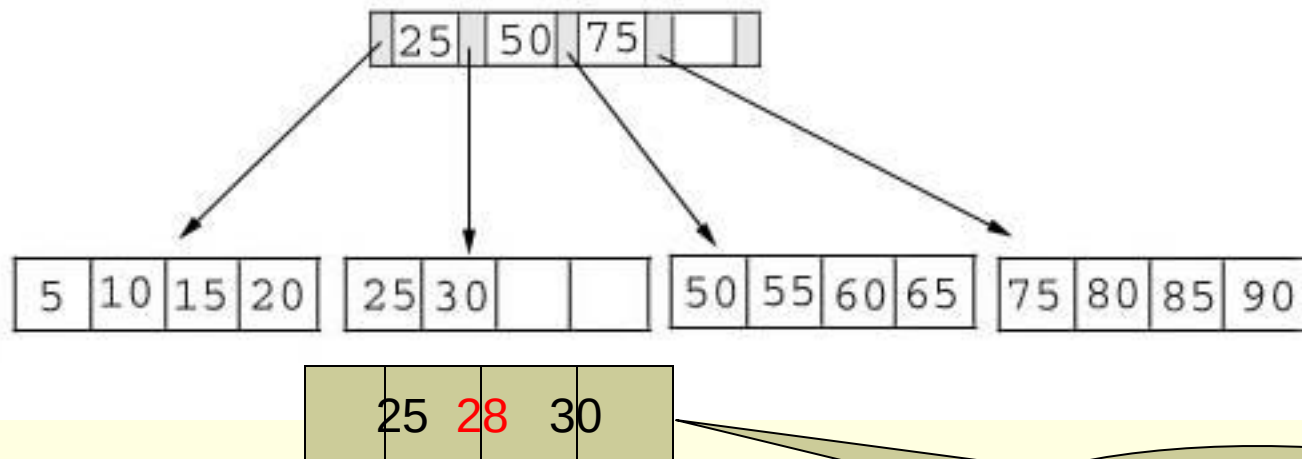
- Result:

1. For the value of 45, not found.
2. For the value of 15, return the position where the pointer located.



# Insertion

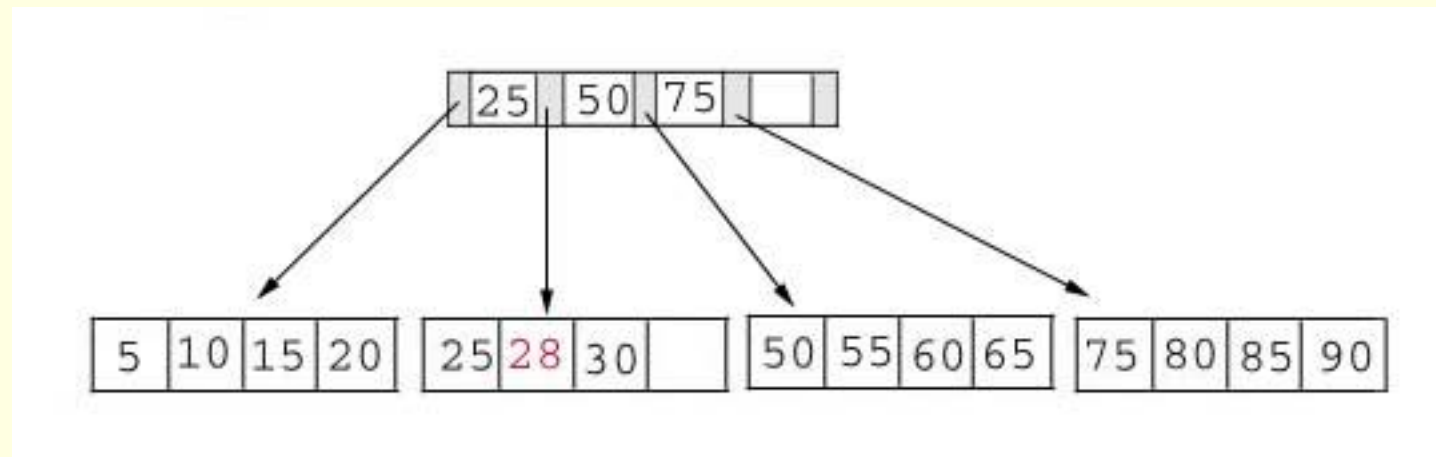
- inserting a value into a B+ tree may unbalance the tree, so rearrange the tree if needed.
- Example #1: insert **28** into the below tree.



Fits inside the  
leaf

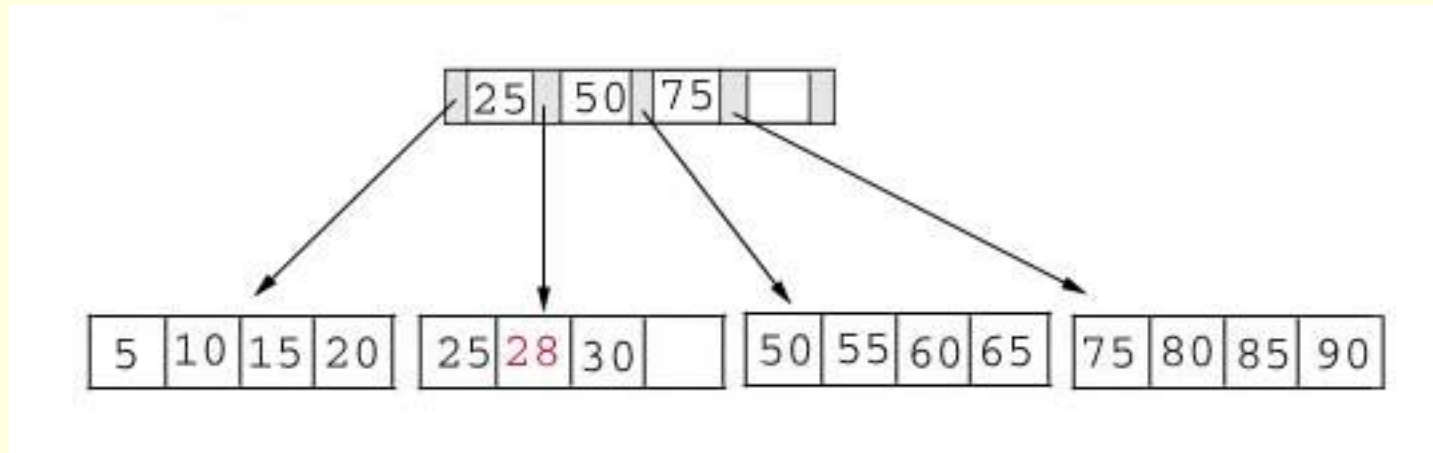
# Insertion

## ■ Result:



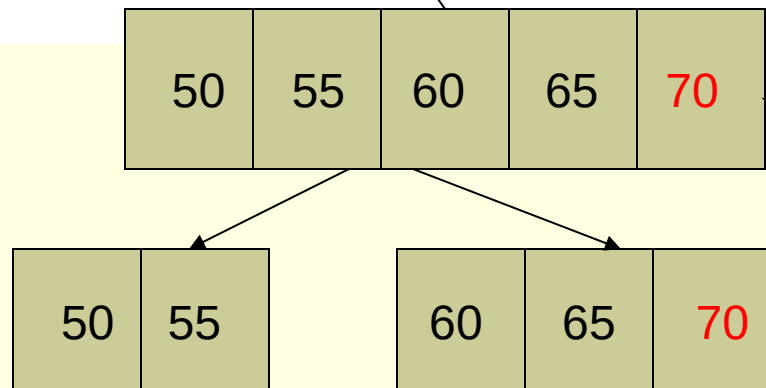
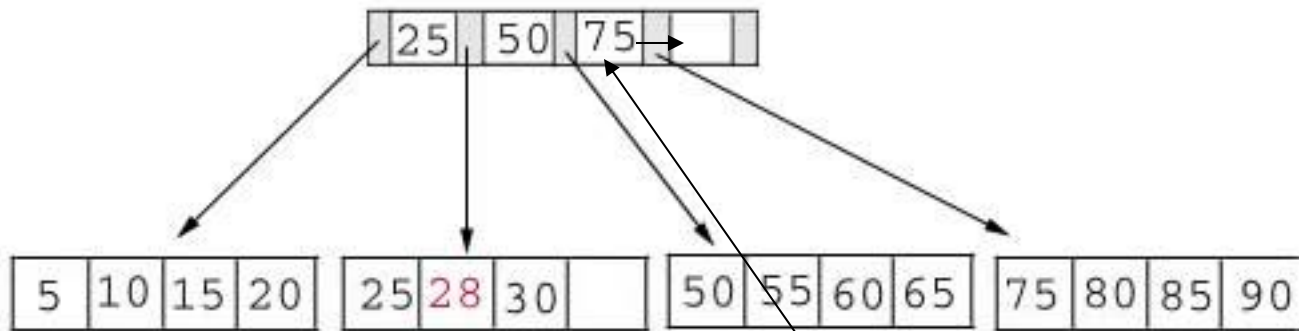
# Insertion

- Example #2: insert **70** into below tree



# Insertion

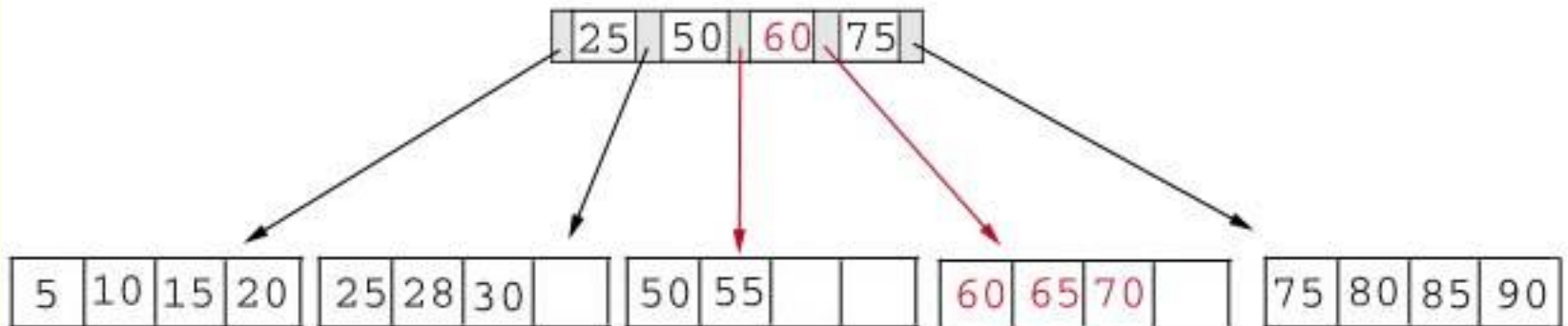
- Process: split the leaf and propagate middle key up the tree



Does not fit  
inside the  
leaf

# Insertion

- Result: chose the middle key 60, and place it in the index page between 50 and 75.



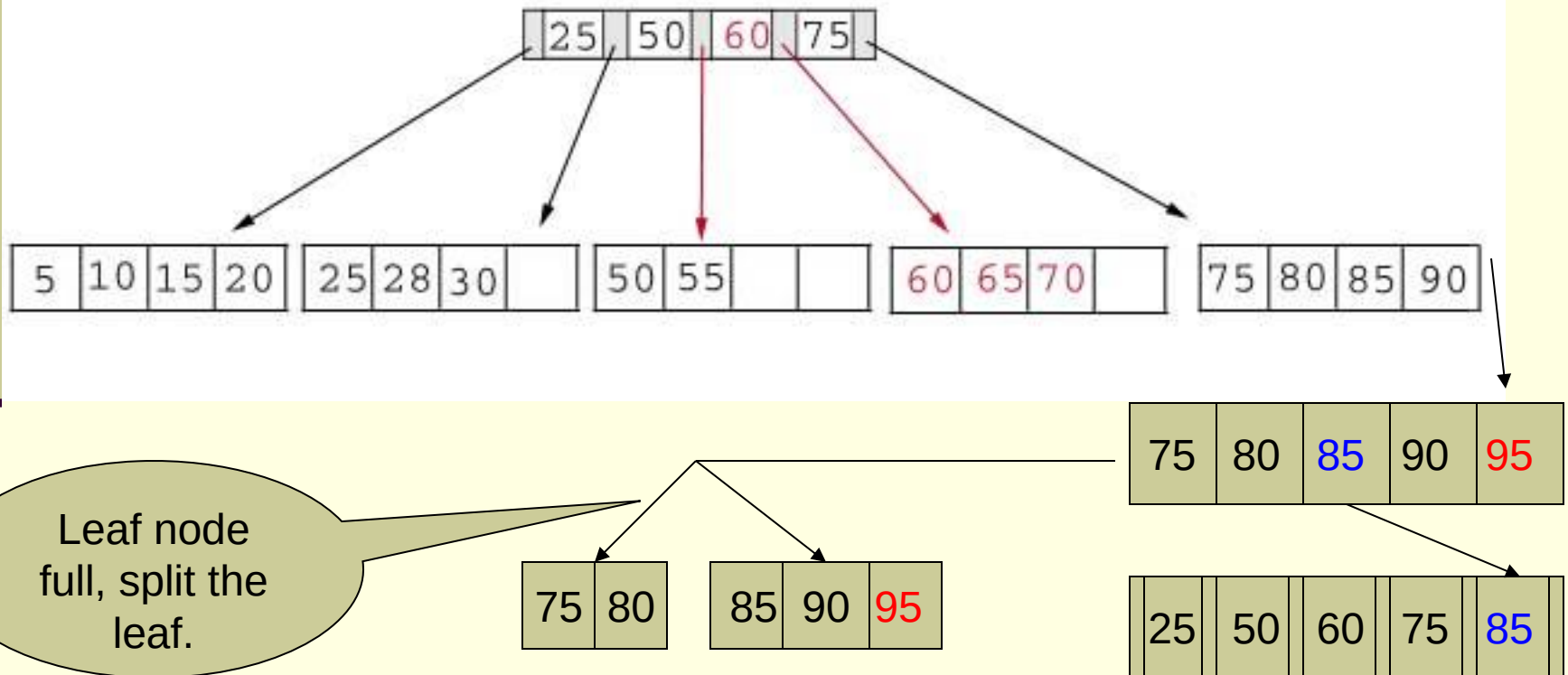
# Insertion

## The insert algorithm for B+ Tree

| Leaf Node Full | Index Node Full | Action                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NO             | NO              | Place the record in sorted position in the appropriate leaf page                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| YES            | NO              | <ol style="list-style-type: none"><li>1. Split the leaf node</li><li>2. Place Middle Key in the index node in sorted order.</li><li>3. Left leaf node contains records with keys below the middle key.</li><li>4. Right leaf node contains records with keys equal to or greater than the middle key.</li></ol>                                                                                                                                                                                                                                                                                                               |
| YES            | YES             | <ol style="list-style-type: none"><li>1. Split the leaf node.</li><li>2. Records with keys <math>&lt;</math> middle key go to the left leaf node.</li><li>3. Records with keys <math>\geq</math> middle key go to the right leaf node.</li></ol> <p>Split the index node.</p> <ol style="list-style-type: none"><li>4. Keys <math>&lt;</math> middle key go to the left index node.</li><li>5. Keys <math>&gt;</math> middle key go to the right index node.</li><li>6. The middle key goes to the next (higher level) index node.</li></ol> <p>IF the next level index node is full, continue splitting the index nodes.</p> |

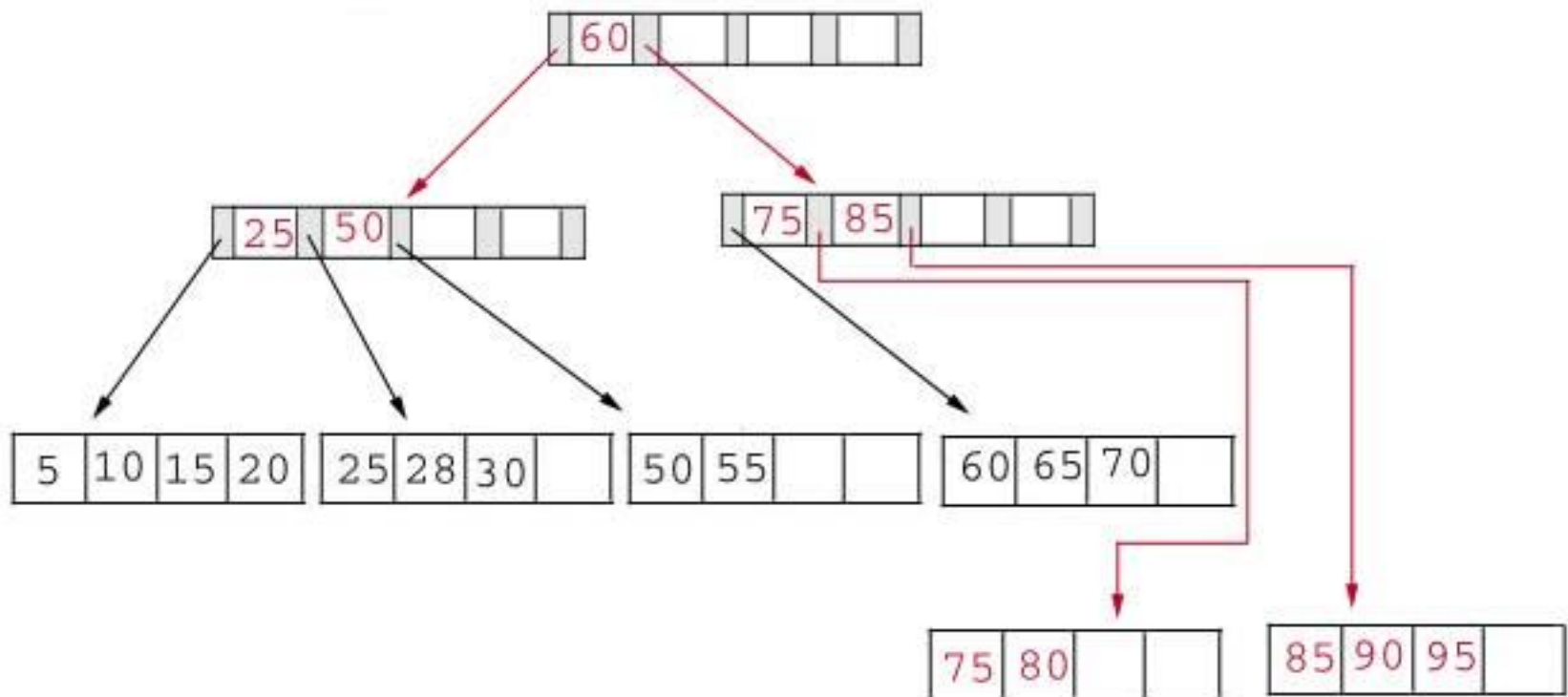
# Insertion

- Exercise: add a key value **95** to the below tree.



# Insertion

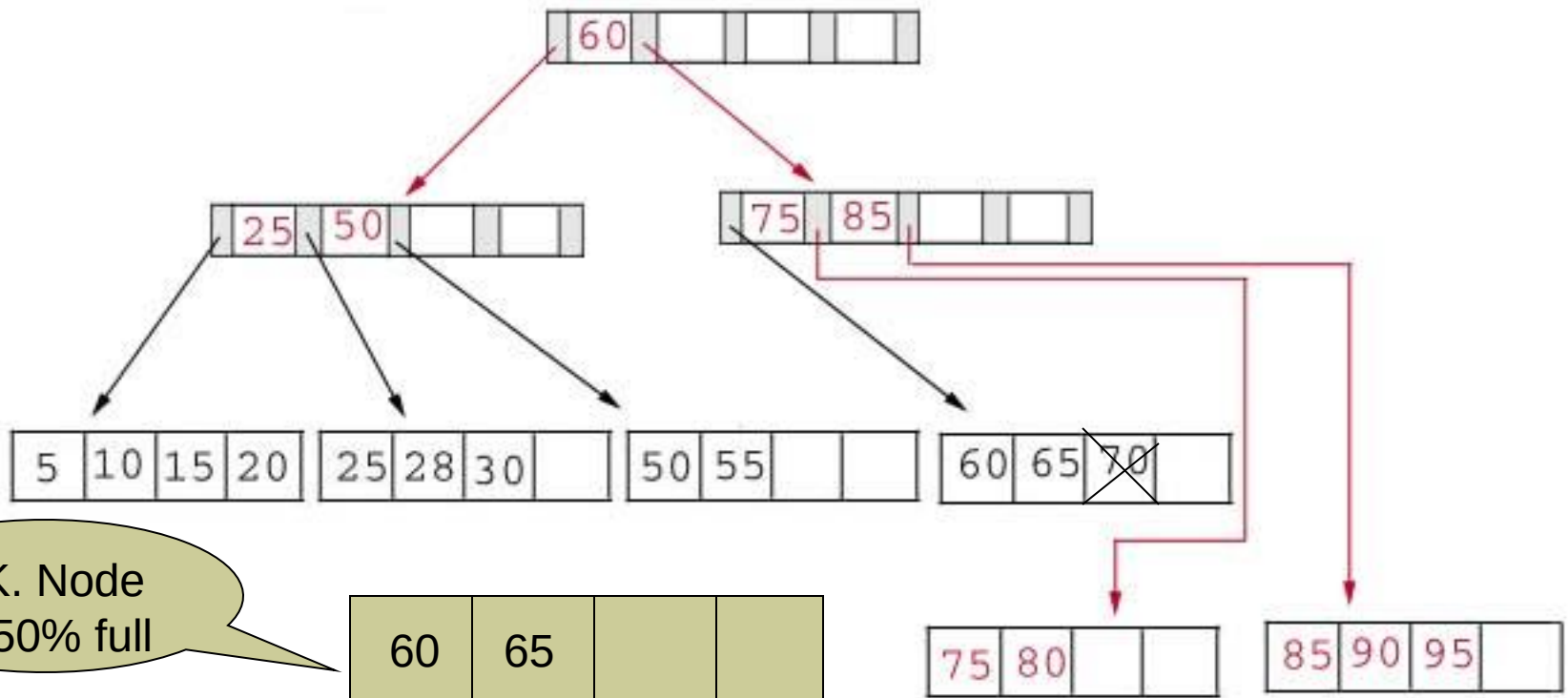
- Result: again put the middle key 60 to the index page and rearrange the tree.





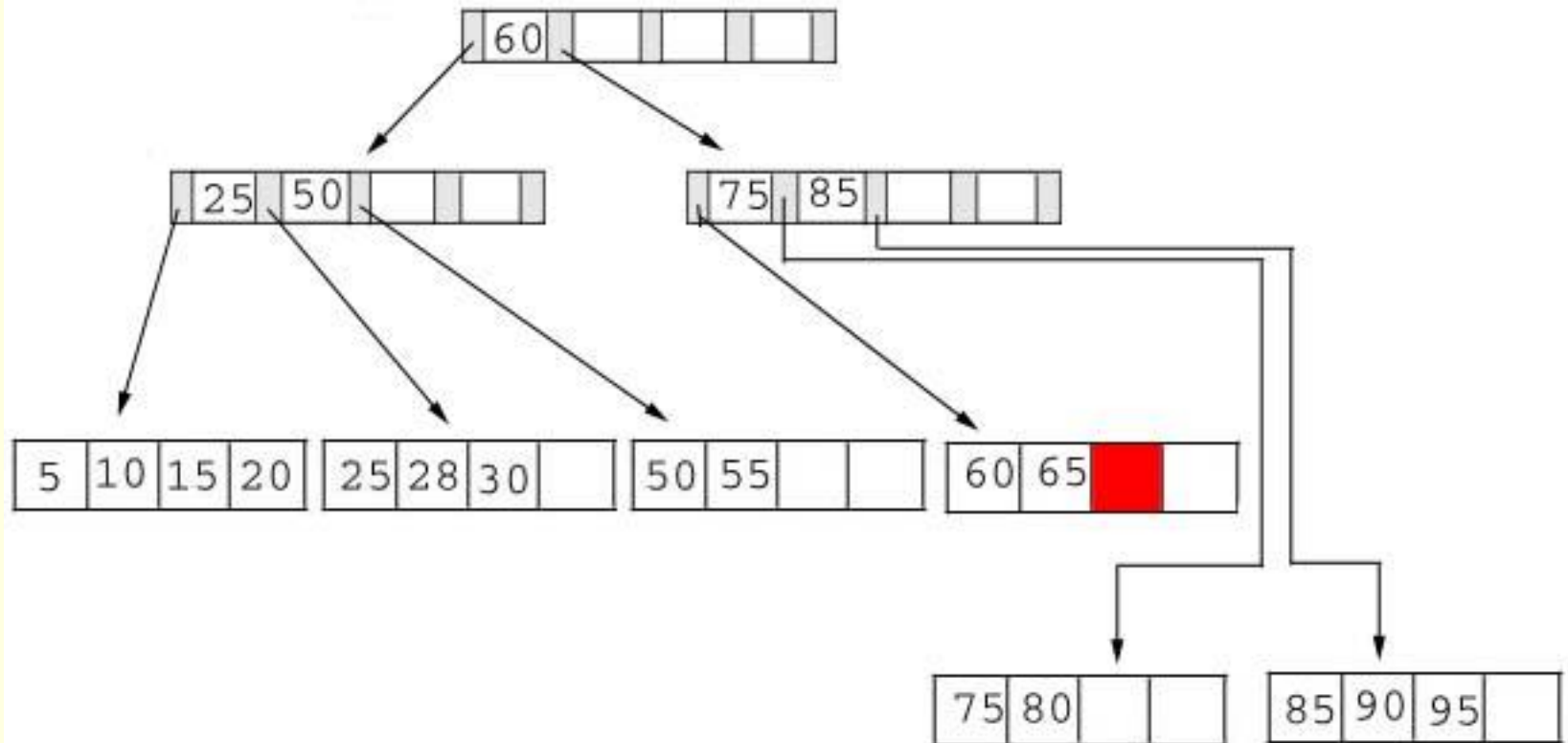
# Deletion

- Same as insertion, the tree has to be rebuilt if the deletion result violate the rule of B+ tree.
- Example #1: delete **70** from the tree



# Deletion

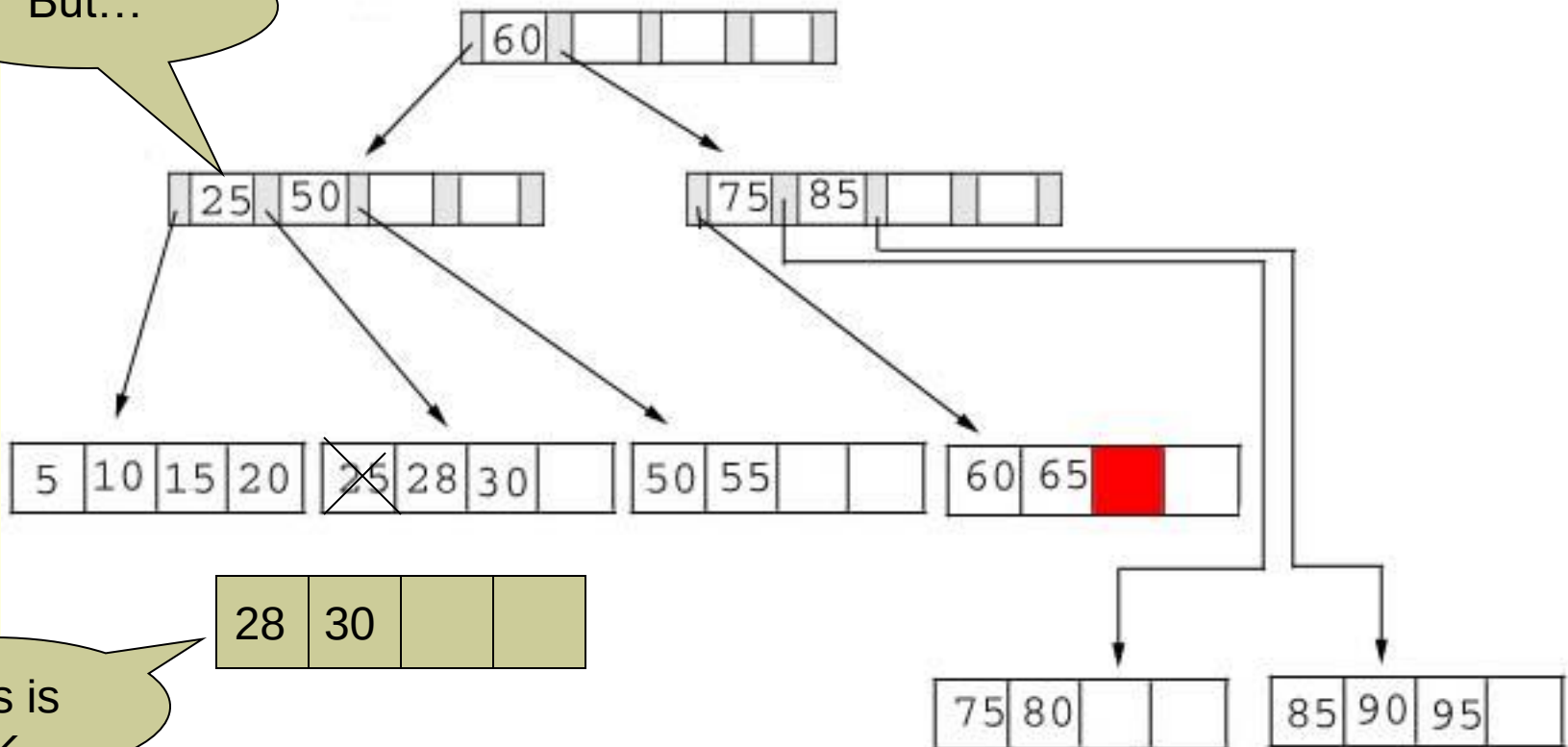
- Result:



# Deletion

Example #2: delete **25** from below tree, but **25** appears in the index page.

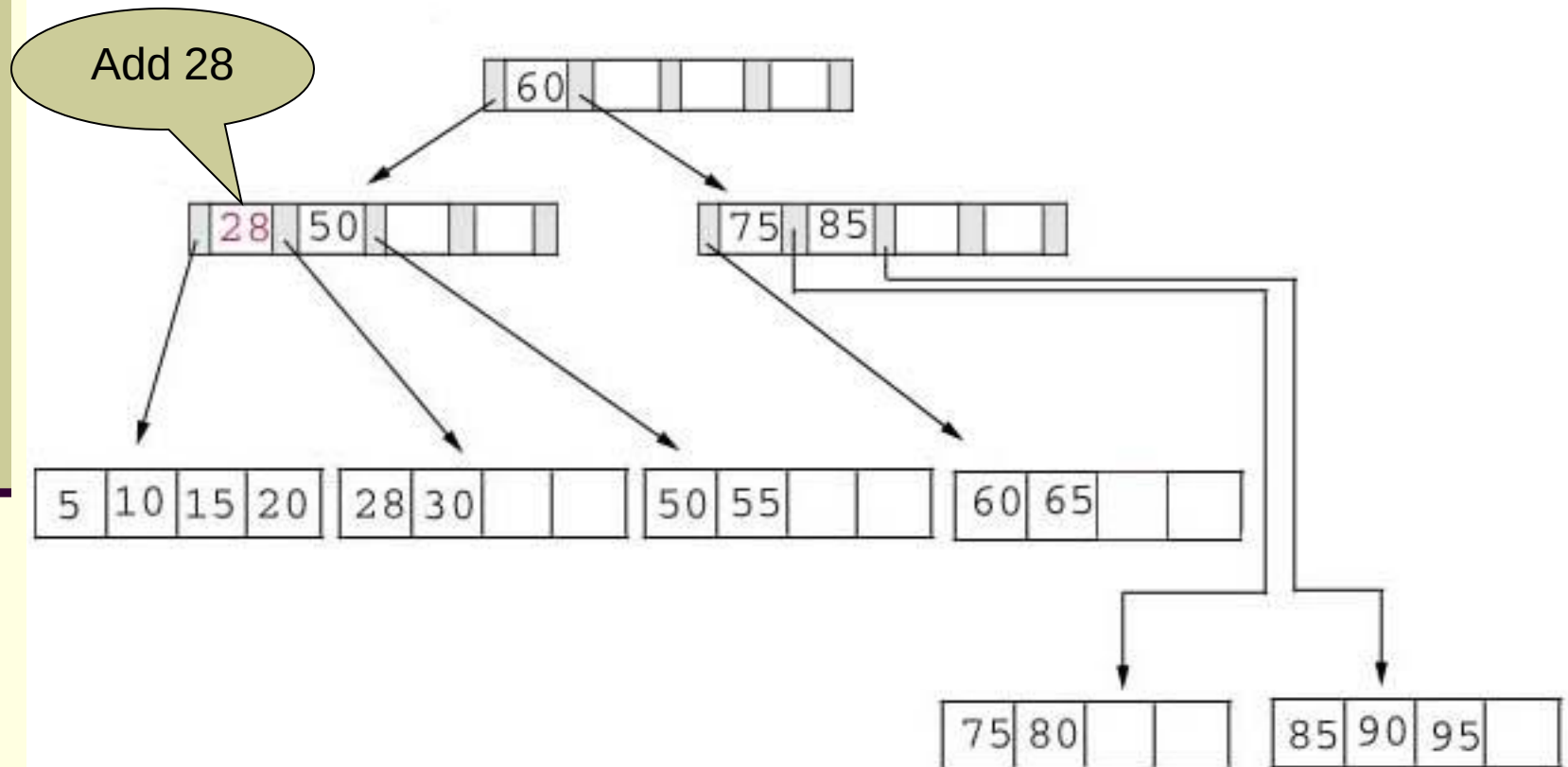
But...



This is  
OK.

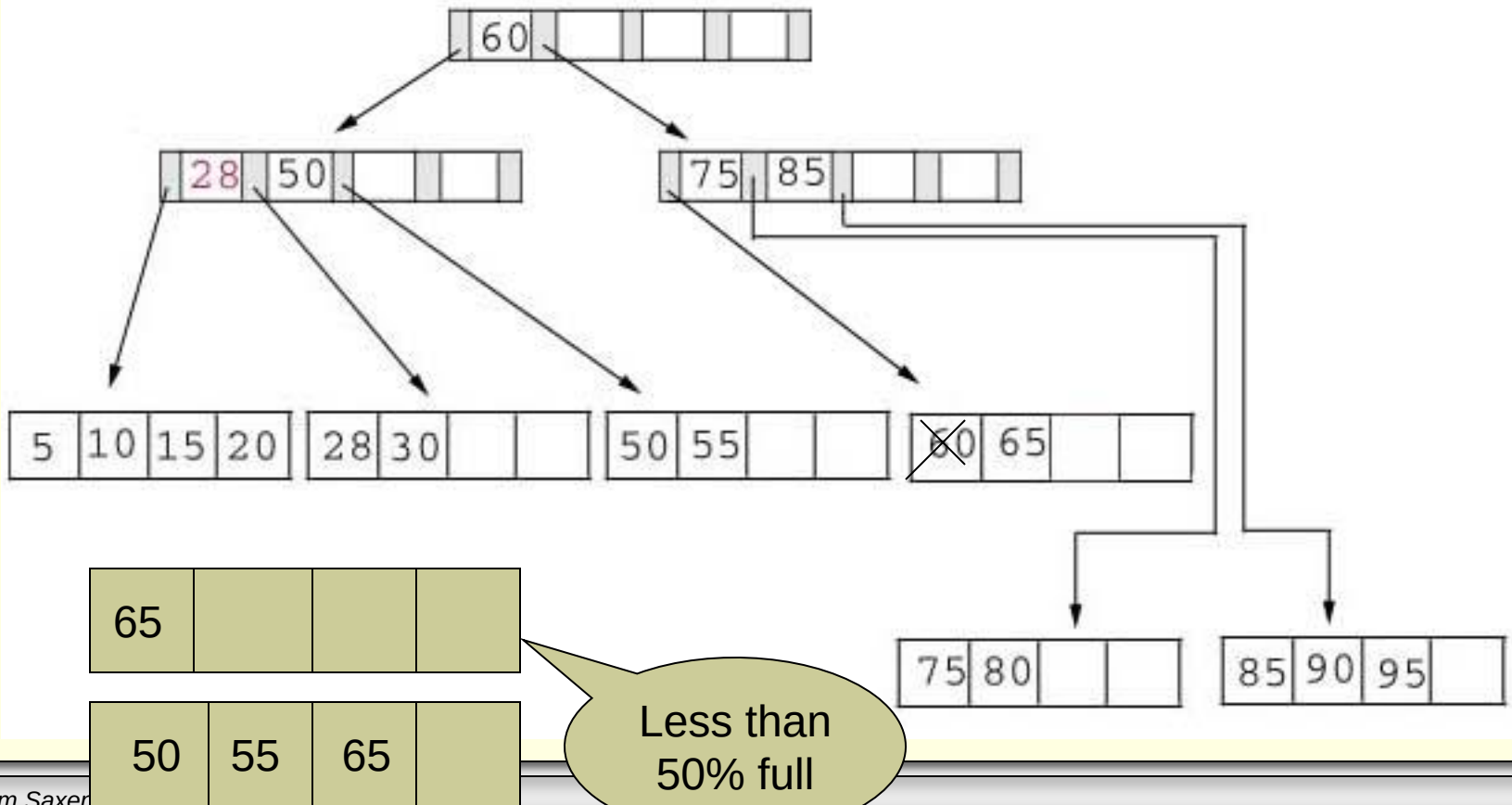
# Deletion

- Result: replace 28 in the index page.



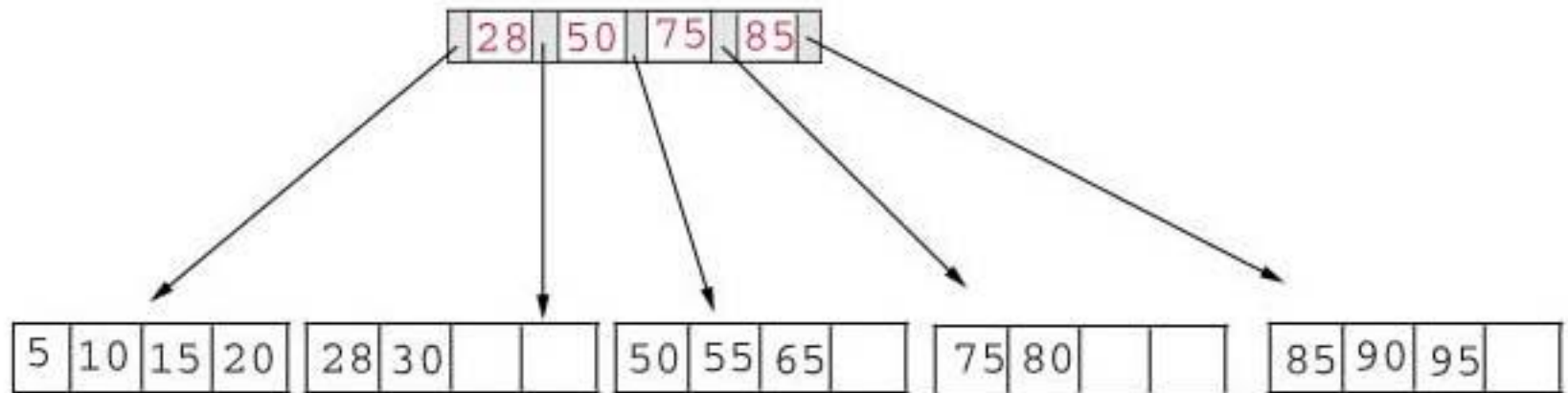
# Deletion

- Example #3: delete 60 from the below tree



# Deletion

- Result: delete 60 from the index page and combine the rest of index pages.



# Deletion

## ■ Delete algorithm for B+ trees

| Data Page Below Fill Factor | Index Page Below Fill Factor | Action                                                                                                                                                                                                                                                                                                                      |
|-----------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NO                          | NO                           | Delete the record from the leaf page. Arrange keys in ascending order to fill void. If the key of the deleted record appears in the index page, use the next key to replace it.                                                                                                                                             |
| YES                         | NO                           | Combine the leaf page and its sibling. Change the index page to reflect the change.                                                                                                                                                                                                                                         |
| YES                         | YES                          | <ol style="list-style-type: none"><li>1. Combine the leaf page and its sibling.</li><li>2. Adjust the index page to reflect the change.</li><li>3. Combine the index page with its sibling.</li></ol> <p>Continue combining index pages until you reach a page with the correct fill factor or you reach the root page.</p> |

# Conclusion

---

- For a B+ Tree:
- It is “easy” to maintain its balance
  - Insert/Deletion complexity  $O(\log_{M/2})$
- The searching time is shorter than most of other types of trees because branching factor is high



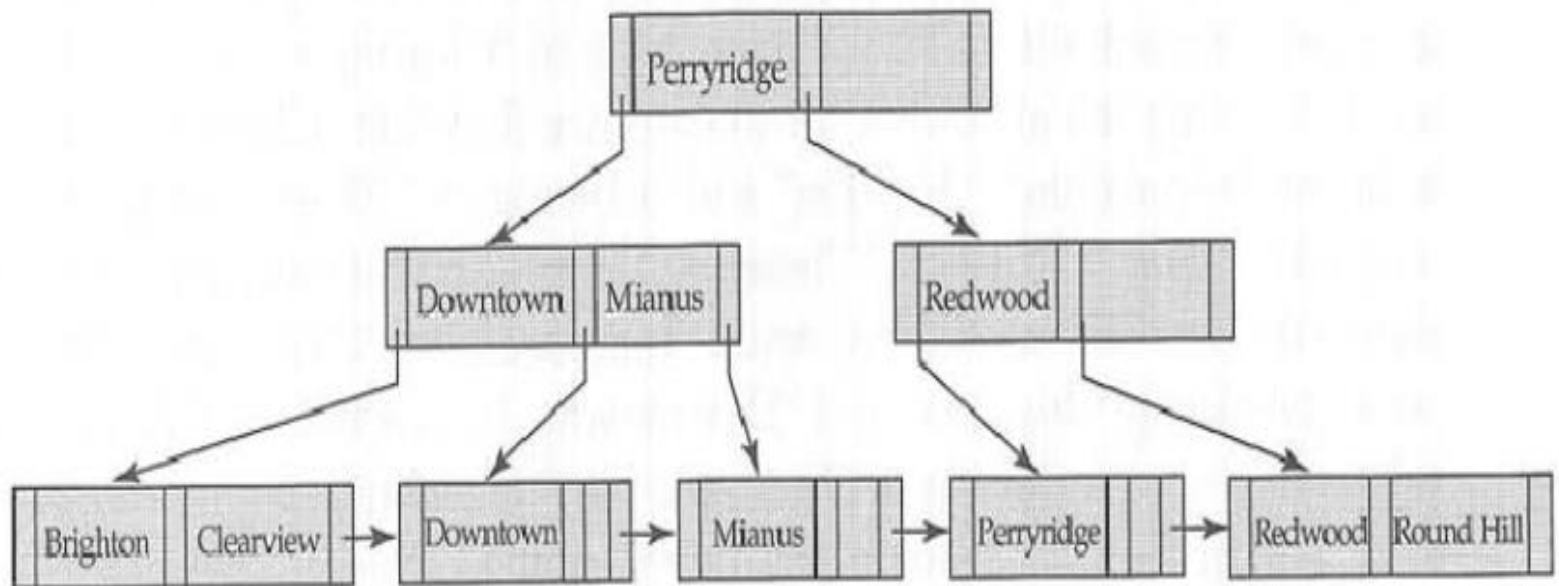
# B+Trees and DBMS

- Used to index primary keys
- Can access records in  $O(\log_{M/2})$  traversals (height of the tree)
- Interior nodes contain Keys only
  - Set node sizes so that the  $M-1$  keys and  $M$  pointers fits inside a single block on disk
    - E.g., block size 4096B, keys 10B, pointers 8 bytes
    - $(8 + (10+8) * (M-1)) = 4096$
    - $M = 228$ ; 2.7 billion nodes in 4 levels
  - One block read per node visited

# B and B+ Tree

1. In a B tree search keys and data stored in internal or leaf nodes. But in B+-tree data store only leaf nodes.
2. Full scan in a B+ tree is very easy because all data are found in leaf nodes. Full scan a B tree requires a full traversal.
3. In a B tree, data may be found in leaf nodes or internal nodes. Deletion of internal nodes is very complicated. In a B+ tree, data is only found in leaf nodes. Deletion of leaf nodes is easy.
4. Insertion in B tree is more complicated than B+ tree.
5. B+ trees store redundant search key but B tree has no redundant value.
6. In a B+ tree, leaf nodes data are ordered as a sequential linked list but in B tree the leaf node cannot be stored using a linked list. Many database systems' implementations prefer the structural simplicity of a B+ tree.

# B + Tree



# B Tree

